Politecnico di Torino

Corso di Laurea Triennale in Ingegneria Gestionale L-9
Anno Accademico 2023/2024
Sessione di Laurea Luglio 2024

Approcci di soluzione per la schedulazione su macchine parallele identiche con un server comune

Relatore:
prof. Rosario Scatamacchia

Candidato:
Simone Martinetti

Sommario

Sommario 1
Indice delle figure 2
Indice delle tabelle 3
Premessa e introduzione al problema 4
Possibili Applicazioni 4
Capitolo 1 - Introduzione 5
1.1 Cos'è la Ricerca Operativa? 5
1.2 La modellizzazione 6
1.3 Schedulazione 8
1.3.1 Una definizione 8
1.3.2 Soluzione e Notazione 9
1.4 Descrizione del problema *P, S1|sj|Cmax*. 11
Capitolo 2 – Modelli e Algoritmi 13
2.1 Modello 1 13
2.2 Modello 2 15
2.3 Algoritmo per determinazione tempo T 16
2.4 Algoritmo di assegnazione 18
Capitolo 3 - Analisi dei risultati computazionali 21
3.1 Istanze e dati in input 21
3.2 Risultati e commento 23
3. 3 Tabelle aggregate 28
3.4 Tabelle estese 29
4 - Conclusioni 42
Bibliografia 43
Appendice A 45
A.1 Codice Modello 1 45
A.2 Codice Modello 2 46
A.3 Codice algoritmo tempo 46

A.4 Codice algoritmo assegnazione 47

Indice delle figure

FIGURA 1: CICLO DELLA MODELLIZZAZIONE 6
FIGURA 2: GANTT CHART 12
FIGURA 3: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 10 31
FIGURA 4: T MEDIO TAB. 10 CAMBIARE 31
FIGURA 5: FO MEDI TAB. 10 31
FIGURA 6: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 10 31
FIGURA 7: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 11 32
FIGURA 8: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 11 32
FIGURA 9: FO MEDIO TAB. 11 32
FIGURA 10: T MEDIO TAB. 11 32
FIGURA 11: T MEDI TAB. 12 33
FIGURA 12: FO MEDI TAB. 12 33
FIGURA 13: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 12 33
FIGURA 14: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 12 33
FIGURA 15: FO MEDI TAB. 13 34
FIGURA 16: T MEDI TAB. 13 34
FIGURA 17 : FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 13 34
FIGURA 18: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 13 34
FIGURA 19: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 14 35
FIGURA 20: T MEDI TAB. 14 35
FIGURA 21: FO MEDI TAB. 14 35
FIGURA 22: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 14 35
FIGURA 23: FO MEDI TAB. 15 36
FIGURA 24: T MEDI TAB. 15 36
FIGURA 25: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 15 36
FIGURA 26: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N. TAB. 15 36
FIGURA 27: FO MEDI TAB. 16 37
FIGURA 28: T MEDI TAB. 16 37
FIGURA 29: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 16 37
FIGURA 30: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 16 37
FIGURA 31: FO MEDI TAB. 17 38
FIGURA 32: T MEDI TAB. 17 38
FIGURA 33: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 17 38
FIGURA 34: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 17 38

FIGURA 35: T MEDI A=0.3 41
FIGURA 36: T MEDI A=0.5 41
FIGURA 37: T MEDI A=0.1 41
FIGURA 38: FO MEDI A=0.3 41
FIGURA 39: FO MEDI A=0.1 41
FIGURA 40: FO MEDI A=0.5 41

Indice delle tabelle

TABELLA 1: DATI PROBLEMA 12
TABELLA 2: NUMERO VARIABILI E NUMERO VINCOLI PER I DUE MODELLI 16
TABELLA 3: TEMPI DI CALCOLO MASSIMI DEL SOLVER IMPOSTATI 21
TABELLA 4: ASSEGNAZIONE 22
TABELLA 5: COMPARAZIONE MODELLI, ALFA =0. 1, M = 2, DATI AGGREGATI PER N E P 28
TABELLA 6: COMPARAZIONE MODELLI, ALFA = 0.1, M = 4, DATI AGGREGATI PER N E P 28
TABELLA 7: COMPARAZIONE MODELLI, ALFA = 0.1, M = 6, DATI AGGREGATI PER N E P 28
TABELLA 8: COMPARAZIONE MODELLI, ALFA = 0.1, M = 6, DATI AGGREGAI PER N E P 29
TABELLA 9: COMPARAZIONE MODELLI, P = 1, DATI AGGREGATI PER N E 29
TABELLA 10: ALFA= 1, M=2, P = 1, 2, 3 MODELLO 1 31
TABELLA 11: ALFA= 1, M=2, P = 1, 2, 3 MODELLO 2 32
TABELLA 12: ALFA= 1, M=4, P = 1, 2, 3 MODELLO 1 33
TABELLA 13: ALFA= 1, M=4, P = 1, 2, 3 MODELLO 2 34
TABELLA 14: ALFA= 1, M=6, P = 1, 2, 3 MODELLO 1 35
TABELLA 15: ALFA= 1, M=6, P = 1, 2, 3 MODELLO 2 36
TABELLA 16: ALFA= 1, M=10, P = 1, 2, 3 MODELLO 1 37
TABELLA 17: ALFA= 1, M=10, P = 1, 2, 3 MODELLO 2 38
TABELLA 18: P = 1, ALFA = 0.3 39
TABELLA 19: P = 1, ALFA = 0.5 40

Premessa e introduzione al problema

Prendere decisioni attive e consapevoli è una parte fondamentale dell'intelletto umano. La Ricerca Operativa è la versione ingegnerizzata di questo processo.

Un problema di Ricerca Operativa viene solitamente tradotto nel linguaggio matematico dell'ottimizzazione, dove si richiede di massimizzare o minimizzare un determinato obiettivo, espresso come una funzione delle variabili decisionali, soddisfacendo allo stesso tempo determinati vincoli.

A seconda delle variabili decisionali, dei vincoli e dell'obiettivo dell'ottimizzazione possiamo discriminare due grandi categorie di modelli: quelli per i cui esistono algoritmi efficienti per trovare la soluzione ottimale e problemi molto più difficili da risolvere in modo ottimale, dove, si accetta una soluzione subottimale che può essere trovata in un tempo ragionevole e con risorse di calcolo limitate, utilizzando algoritmi specifici per casi, chiamati euristiche.

Il problema che verrà affrontato riguarda la pianificazione di lavori indipendenti su macchine parallele con un server comune per le operazioni di setup, l'obiettivo del quale è minimizzare il tempo di completamento totale. In questo caso, il server comune è responsabile dell'esecuzione delle operazioni di setup e, pertanto, non devono esserci conflitti durante il loro svolgimento.

In questo elaborato verranno considerate e valutate due formulazioni (modelli matematici, (Elidrissi A. R., 2021)) esatte alternative indicizzate nel tempo per il problema. Inoltre, verranno presentati due algoritmi che si pongono di migliorare le prestazioni delle due formulazioni. Infine, si valuteranno i risultati ottenuti attraverso vari set di dati che arrivano a coinvolgere fino a 100 lavorazioni e 10 macchine parallele.

Possibili Applicazioni

Le applicazioni di questa fattispecie di schedulazione si possono trovare nei sistemi di produzione flessibili in cui diversi centri di lavorazione condividono un robot per eseguire funzioni di cambio utensile (Koulamas, 2009). Nelle unità di produzione di settori industriali come l'industria dei semiconduttori (Kim M.-Y. Y., 2012) e l'industria della produzione di componentistica in plastica (Bektur, 2019) sono utilizzati server comuni per eseguire le attività di ispezione nel primo caso, mentre nel secondo sono utilizzate per eseguire le funzioni di cambio stampo e pulizia. Nella produzione cellulare (Jiang, 2015), un server può essere utilizzato per caricare e scaricare materiali tra le stazioni di lavorazione. Nella gestione della catena di approvvigionamento, la presenza di un server comune si riscontra nella logistica di terzi e nel carico dei camion (Liu, 2019). Un altro esempio si presenta nel computing di rete, dove un server di rete configura le workstation caricando il software richiesto (Glass, 2020).

Capitolo 1 - Introduzione

1.1 Cos'è la Ricerca Operativa?

Per poter comprendere cosa è la Ricerca Operativa oggi è opportuno conoscere qualcosa della sua storia passata.

È sostanzialmente riconosciuto che la Ricerca Operativa (abbreviata RO) sia nata durante la II Guerra Mondiale, per risolvere, da parte delle forze alleate, complessi problemi tattici e strategici, impossibili da risolvere ricorrendo alle discipline sino ad allora esistenti. Vennero, quindi, organizzati gruppi di scienziati, matematici, economisti, sociologi, strateghi ecc., che affrontarono con metodi nuovi problemi quali la localizzazione ottima dei radar, il controllo del fuoco della contraerea, l'individuazione dei sottomarini nemici ecc.

Finita la guerra, si pensò che l'approccio messo a punto sino a quel momento poteva essere utilizzato anche per risolvere complessi problemi di natura civile. Alcuni scienziati, ritornati nelle loro Università, cercarono di sviluppare un fondamento teorico delle tecniche che avevano empiricamente sviluppato in precedenza, mentre altri si dedicarono a sviluppare nuove tecniche.

Le prime applicazioni civili della Ricerca Operativa furono nell'ambito della pianificazione della produzione dell'industria del petrolio. Infatti, all'inizio, solamente le grandi industrie potevano permettersi il mantenimento al loro interno di gruppi di specialisti in Ricerca Operativa. Successivamente, quando le categorie dei problemi risolvibili con tali metodi furono chiaramente individuate e le tecniche diventarono standardizzate, anche le medio-piccole imprese si affacciarono al mondo della RO, senza dover investire eccessivi capitali in ricerca e sviluppo.

Nel settore dei servizi e della pubblica amministrazione, le prime applicazioni della Ricerca Operativa comparvero alla metà degli anni '60. Oggigiorno, servizi quali banche, ospedali, supermercati, aziende di trasporto ecc., riconoscono la necessità di utilizzare metodi di RO per aumentare l'efficienza e l'efficacia nella fornitura dei propri servizi, in particolare per redigere piani di trasporto, pianificare la localizzazione e il dimensionamento di ospedali e scuole, programmare il sistema delle comunicazioni ecc.

Lo sviluppo e l'affermarsi della RO sono stati e sono tuttora fortemente aiutati dalla disponibilità di computer sempre più potenti e veloci. Dal canto suo, la Ricerca Operativa ha contribuito e continua a contribuire allo sviluppo delle discipline informatiche, si pensi, ad esempio, alla progettazione di reti e computer o alla diagnostica degli errori nei circuiti elettronici, od ancora allo studio della complessità computazionale od alla costruzione di algoritmi di calcolo.

Concludendo, non esiste una definizione univoca di Ricerca Operativa. Per alcuni si tratta di quell'insieme di problemi, metodi e soluzioni che sono stati annoverati sotto il nome di Ricerca Operativa negli ultimi 50 anni. Per altri è un'attività, un processo, caratterizzata da un approccio sistemico, per sua natura applicativo.

Potremmo per semplicità definirla così:
"Ricerca Operativa è l'applicazione di metodi scientifici per la soluzione di problemi complessi che nascono nella direzione e gestioni di grandi sistemi di uomini, macchine, materiali e denaro

nell'industria, affari, amministrazione e difesa. L'approccio caratteristico della ricerca operativa è lo sviluppo di un modello matematico del sistema allo studio, che incorpora la misurazione di fattori di casualità e di rischio, mediante il quale predire e confrontare i risultati di decisioni, strategie e controlli alternativi. Lo scopo consiste nell'aiutare i decisori a determinare scientificamente la loro politica e le loro azioni".

Oppure con quest'altra affermazione:
"Ricerca Operativa è relativa al decidere scientificamente come progettare ed utilizzare al meglio i sistemi uomo-macchina, solitamente sotto condizioni di utilizzo e di risorse scarse".

Entrambe le definizioni, sebbene incomplete, sottolineano l'aspetto fondamentale della Ricerca Operativa, quello di essere una disciplina per l'aiuto alla decisione nel trattamento di problemi complessi del mondo reale. Inoltre, la prima definizione, introduce ed individua quello che è l'approccio caratteristico della disciplina, ovvero quello dello sviluppo di un modello matematico, che cercheremo brevemente di riassumere nel successivo paragrafo (Della Croce, 2010).

1.2 La modellizzazione

I modelli di programmazione matematica sono modelli che descrivono le caratteristiche della soluzione ottima di un problema di ottimizzazione attraverso relazioni matematiche. Un modello di programmazione matematica è composto dai seguenti elementi:

- **Insiemi**: raggruppano gli elementi del sistema;
- **Parametri**: sono i dati del problema e rappresentano delle quantità fissate che dipendono dai diversi elementi del sistema;
- **Variabili decisionali o di controllo**: sono le grandezze del sistema di cui non si conosce il valore (assimilabili a delle incognite) e sulle quali si può agire per determinare diverse soluzioni alternative del problema;
- **Vincoli**: sono delle relazioni matematiche che descrivono le condizioni di ammissibilità delle soluzioni. Servono quindi per discriminare le combinazioni di valori delle variabili decisionali che rappresentano soluzioni accettabili del problema, da quelle che non lo sono;
- **Funzione obiettivo**: è la quantità da massimizzare o minimizzare, espressa come funzione delle variabili decisionali.

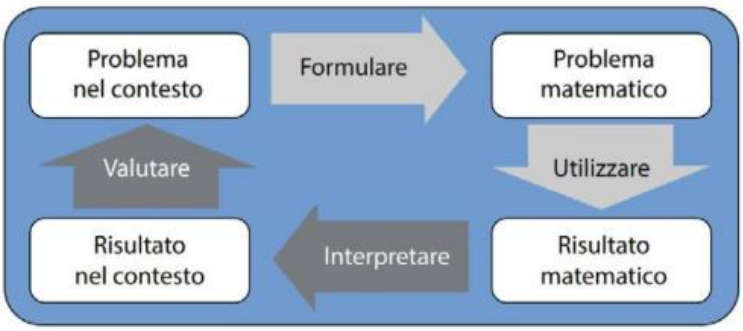


FIGURA 1: CICLO DELLA MODELLIZZAZIONE

Un modello di programmazione matematica dichiara le caratteristiche della soluzione cercata (che cosa), piuttosto che definire la strategia per la ricerca della soluzione stessa (come). Un modello di programmazione matematica potrebbe essere paragonato ad un linguaggio dichiarativo, che indica cosa si vuole ottenere, piuttosto che a un linguaggio procedurale, che indica i passi per ottenere il risultato cercato.

Per schematizzare il processo (Figura 1) il primo passo è la costruzione del modello astratto partendo dalla fattispecie reale (in modo da renderlo utilizzabile per ogni problema reale dello stesso tipo, a prescindere dai parametri del singolo contesto). Questo passo è il più delicato perché richiede esperienza, immaginazione e creatività. Inoltre, bisogna decidere quali aspetti della fattispecie reale considerare nel modello (ovvero quanto semplificare nel passaggio reale-astratto). Quindi non esiste un modo ritenuto giusto ed univoco per la formulazione di un modello. Possono esistere per lo stesso problema reale due modelli completamente diversi. La seconda fase del processo consiste nella costruzione delle soluzioni del modello. Questa parte richiede l'utilizzo di tecniche risolutive (algoritmi o programmi di calcolo). In questa fase non è presente soggettività, essa si basa infatti su rigorosi metodi matematici. Gli ultimi passi consistono nell'interpretazione e nella valutazione dei risultati del modello applicato a quel problema reale. Occorre soggettività e attenzione nella traduzione da modello a realtà a causa dei possibili effetti creati dalle semplificazioni in fase di formulazione. Infatti, alcuni elementi trascurati possono rivelarsi fondamentali e quindi le soluzioni trovate diventerebbero completamente sbagliate. In questo caso bisognerà riniziare con una formulazione di un nuovo modello che tenga conto anche di questi fattori.

La risoluzione di un problema di ottimizzazione formulato con un modello di programmazione matematica consiste nella determinazione dei valori delle variabili che soddisfano tutti i vincoli (soluzioni ammissibili) e massimizzano o minimizzano il valore della funzione obiettivo (soluzione ottima). Come nei linguaggi dichiarativi, una volta messo a punto il modello matematico, la ricerca della soluzione ottima può essere effettuata da appositi motori di ottimizzazione; pertanto, un modello di programmazione matematica non ha solo valenza descrittiva, ma anche operativa. La soluzione oltre che ottima o ammissibile può anche risultare inammissibile se non si rispettano i vincoli imposti.

I modelli di programmazione lineare sono una particolare classe di modelli di programmazione matematica in cui la funzione obiettivo è un'espressione lineare delle variabili decisionali.

I vincoli sono determinati da un sistema di equazioni e/o disequazioni lineari.

La formulazione generale di un modello di Programmazione Lineare è la seguente:

$\min(max)\ z \quad = c_1x_1 + \ c_2x_2 + \cdots + \ c_jx_j + \cdots + \ c_nx_n \ \ (+cost.) \quad FUNZIONE\ OBBIETTIVO$

$soggetto\ a\ (s.t.)\ VINCOLI$:

$$a_{11}x_1 + \ a_{12}x_2 + \cdots + \ a_{1j}x_j + \cdots + \ a_{1n}x_n \ \leq (\geq, =) \quad b1$$

$$a_{21}x_1 + \ a_{22}x_2 + \cdots + \ a_{2j}x_j + \cdots + \ a_{2n}x_n \ \leq (\geq, =) \quad b2$$

$$\ldots$$

$$a_{m1}x_1 + \ a_{m2}x_2 + \cdots + \ a_{mj}x_j + \cdots + \ a_{mn}x_n \ \leq (\geq, =) \quad bm$$

$DOMINIO\ VARIABILI$:

$$x_j \ \in \mathbb{R}^+ \ (\mathbb{Z}^+) \ \ \forall\, j = 1, \ldots\ldots, n$$

Dove:

- z è la funzione obiettivo da minimizzare (min) o massimizzare (max);

- $x_j \; \forall j = 1, \dots . n$ sono le variabili decisionali (incognite) che possono essere reali (eventualmente non negative), intere (eventualmente non negative) o binarie ($x_j \in \{0, 1\}$);
- $c_j \; \forall j = 1, \dots . n$ sono i coefficienti di costo (min) o profitto (max) e sono costanti note del problema (parametri);
- $a_{ij} \; \forall j = 1, \dots . n \;\; \forall i = 1, \dots , m$ sono i coefficienti tecnologici (parametri);
- $b_i \; \forall i = 1, \dots . m$ sono i termini noti (parametri).

Ovviamente, ci possono essere più tipologie di variabili e sia per le incognite sia per i parametri anche più di due indici, per esempio se si aggiunge anche l'indice t si introduce il tempo, il time bucket (unità di tempo minima ed indivisibile), che renderà il modello adatto a descrivere un problema o una situazione su più unità di tempo (per esempio per giorni, settimane, mesi, anni, …).

Da notare che aumentando il numero di indici e variabili e parametri da considerare nel modello aumenta la difficoltà di "astrazione" del problema e di risoluzione di un eventuale solver.

In base alla natura o dominio delle variabili decisionali, si parla di:

- **modelli di Programmazione Lineare** (in senso stretto, PL) se tutte le variabili possono assumere valori reali;
- **modelli di Programmazione Lineare Intera** (PLI) se tutte le variabili possono assumere valori interi;
- **modelli di Programmazione Lineare Intera Mista** (PLIM o MIP) se alcune variabili possono assumere valori reali e altre valori interi.

L'importanza dei modelli di programmazione lineare risiede nella loro notevole valenza operativa (a scapito a volte della potenza descrittiva), grazie alla disponibilità di motori di ottimizzazione particolarmente efficienti[1].

1.3 Schedulazione

1.3.1 Una definizione

La schedulazione (o scheduling) è uno degli argomenti più ampiamente studiati nel campo della ricerca operativa, che comprende una vasta varietà di problemi (Potts, 2009).

La schedulazione è un processo di decisione che si pone in svariati contesti applicativi quali i sistemi manifatturieri, sistemi di elaborazione, sistemi di distribuzione e trasporti ed altri sistemi di servizio; ed è uno dei temi più ampiamente studiati nel campo della ricerca operativa e consiste in una vasta classe di problemi, molto diversi tra loro per complessità e struttura.

Si definiscono problemi di scheduling tutti quei problemi decisionali in cui riveste importanza il fattore tempo, visto come risorsa scarsa da allocare in modo ottimo a determinate attività (Alfieri, 2023).

Esempi di problemi di schedulazione possono essere i seguenti:

[1] Ulteriori approfondimenti e alcuni esempi di modelli sono disponibili online al sito: https://www.math.unipd.it/~luigi/courses/ricop/m01.modPL.01.modelli.pdf

- Un'azienda produce sacchetti di carta. Il processo produttivo consiste di tre stadi ed in ogni stadio l'operazione è effettuata da un operaio. Ogni ordine di produzione consiste di un certo numero di sacchetti di un certo tipo che devono essere consegnati al cliente entro una certa data. Il costo legato ad un eventuale ritardo rispetto alla data di consegna dipende dall'entità del ritardo e dall'importanza dell'ordine (ovvero del cliente). Conoscendo i tempi richiesti dalle varie operazioni per ciascun ordine, un obiettivo della pianificazione è quello di organizzare le lavorazioni in modo tale da minimizzare queste penali.
- Nell'industria meccanica, i centri di lavorazione devono effettuare lavorazioni (taglio, fresatura, tornitura) su vari pezzi che vengono montati sui centri stessi. Diverse operazioni richiedono tempi diversi, e/o diversi tipi di utensile, il che può comportare un certo tempo di riconfigurazione da parte delle macchine stesse. Il problema può consistere nel determinare l'ordinamento dei pezzi in modo da terminare tutte le lavorazioni il prima possibile.

Questo fa capire come cambiando minimamente le condizioni di "utilizzo" (setup, macchine, sequenze sulle macchine…) cambi completamente l'astrazione del problema, e questo fa sì che la schedulazione comprenda un campo molto ampio di problemi diversi e di diverse complessità.

Diversi autori hanno tentato una sistematizzazione e una sintesi di tali problemi e ciò ha consentito di classificare molti modelli in modo organico e di unificare alcuni approcci algoritmici. Tuttavia, per i problemi di scheduling più difficili, non è possibile indicare un unico approccio nettamente preferibile, rispetto agli altri, per la loro soluzione, uno di questi è per esempio il caso che verrà trattato in questo elaborato.

Inoltre, più del 90% della letteratura sui problemi di schedulazione ignora i tempi o i costi di setup (Allahverdi, 2007). Tuttavia, gli studi che coinvolgono tali operazioni sono altamente pertinenti perché sono frequentemente identificati in scenari in cui la produttività è significativamente influenzata non solo dai tempi di setup, ma anche dalle risorse di setup (Silva, 2021).

1.3.2 Soluzione e Notazione

I problemi di schedulazione sono problemi concettualmente semplici ma difficili da un punto di vista di tempi di calcolo poiché, per raggiungere l'ottimo, spesso è necessaria una enumerazione completa. Se si hanno ad esempio 20 job, le sequenze possibili sono 20! (ovvero $2{,}43*10^{18}$); se si aumentano il numero dei job di 1, le sequenze possibili diventano $21*2{,}43*10^{18}$. Se si fosse capaci di generare un milione di sequenze al secondo, per generare tutte le sequenze si impiegherebbero $2{,}43*10^{12}$ secondi, ovvero 77146 anni. È quindi evidente che una enumerazione completa di tutte le sequenze non è possibile ma occorre sviluppare metodi di soluzione più efficienti. I metodi di soluzione si distinguono, come si è già accennato, in esatti (riescono a trovare la soluzione ottima) ed euristici (si accontentano di una buona soluzione).

Si affrontano ora nel dettaglio quelli che sono gli elementi di un problema di schedulazione.

Risorse ed attività possono venire indicate con i termini **macchina** e **job**. Nel seguito si indicherà con **m** e **n** il numero di macchine e di job rispettivamente.

Esistono varie informazioni che possono essere associate a un job, le quali contribuiscono alla definizione del problema di scheduling:

- **Tempi di setup**. Indica il tempo s_j necessario per riattrezzare la macchina se si vuole fare seguire la lavorazione del job k a quella del job j;
- **Tempo di processo**. Indica il tempo p_j necessario a processare il job j su una delle risorse.
- **Preemption**. La preemption indica la possibilità di interrompere la lavorazione di un job per lavorare un job diverso più urgente (con w_j maggiore);
- **Vincoli di precedenza**. In molti casi esistono vincoli di precedenza tra task di un job o tra diversi job;
- **Tempo di processamento** del job j (t_j). Viene definito come $t_j = s_j + p_j$ (vedi seguito);
- **Tempo di rilascio** (release date) r_j. Indica l'istante di tempo in cui è possibile iniziare l'esecuzione del job j;
- **Tempo di consegna** (due date) d_j. Indica l'istante di tempo entro il quale l'esecuzione del job j dovrebbe essere terminata;
- **Peso** w_j. Rappresenta l'importanza relativa del job j rispetto agli altri.

Oltre alle caratteristiche relative ai job, vi sono quelle relative al sistema che rappresentano il modo in cui le risorse (macchine) sono tra loro collegate all'interno del sistema di produzione:

- **Macchina singola**. In questo caso i job richiedono tutti la stessa risorsa per essere eseguiti. Se esistono più macchine, si ha il caso di macchine parallele (identiche o meno) ed i job devono passare su una sola di esse.
- **Macchine singole parallele**. Come macchina singola, solo che ce n'è più di una.
- **Flow shop**. In questo caso il sistema consiste di m macchine disposte in serie e ciascun job deve essere eseguito da ciascuna delle m macchine successivamente. Parliamo di flow shop puro se tutti i job passano su tutte le macchine della linea mentre parliamo di flow shop generale se i job possono passare solo su alcune macchine della linea.
- **Job shop**. Anche qui vi sono m macchine, ma ciascun job può visitare un diverso insieme di macchine (al limite tutte le m macchine) in un ordine diverso da ogni altro job nel sistema.

La soluzione di un problema di scheduling prende il nome di **schedule**. In termini generali, uno schedule è una descrizione completa dell'utilizzo temporale delle macchine da parte dei job che devono essere eseguiti. Il concetto di sequenza è diverso da quello di schedule. La sequenza, infatti, specifica solo l'ordine in cui i job devono essere eseguiti da ciascuna macchina, mentre lo schedule ne specifica anche gli istanti d'inizio, ovvero, lo schedule è una sequenza temporizzata. Se, oltre alla temporizzazione, si ha anche l'indicazione della risorsa (work centre), allora si parla di dispatching list. Ovviamente, si è interessati solo a schedule ammissibili (che rispettino i vincoli imposti). La rappresentazione di una schedule può essere fatta tramite il cosiddetto diagramma di Gantt, un grafico che rappresenta su un asse temporale le varie attività[2].

Infine, gli obiettivi di un problema di scheduling possono essere diversi. Per esprimerli formalmente, occorre introdurre alcune grandezze associate ai vari job in uno schedule ammissibile:

[2] È uno "strumento" facile da utilizzare e permette di ottenere risultati visibili, ma non è efficace se ci sono molti job perché in questo caso si perde il vantaggio della visibilità; inoltre è solamente uno strumento descrittivo, permette cioè di rappresentare graficamente una soluzione ma di per sé non genera alcuna soluzione (Figura 2).

- **Tempo di completamento C_j**. È l'istante in cui l'ultimo task del job j (e quindi l'intero job j) termina, misurato a partire dall'istante 0.
- **Lateness L_j.** È la differenza tra il tempo di completamento e la data di consegna del job j. Si noti che se `e positiva, la lateness indica un ritardo, se negativa un anticipo rispetto alla due date. Si ha dunque $L_j = C_j - d_j$.
- **Tardiness T_j.** Coincide con la lateness quando questa è positiva, ed è zero altrimenti, ossia Tj = max (0, L_j).
- **Earliness E_j**. Coincide con la lateness quando questa è negativa, ed è zero altrimenti, ossia E_j = max (0, $-L_j$).
- **Flow time F_j**. È la differenza tra il tempo di completamento e la data di rilascio, $F_j = C_j - r_j$.

Utilizzando queste grandezze è possibile costruire varie funzioni obiettivo, tra cui si citano:

- **Massimo tempo di completamento o makespan Cmax**. Cmax = $\max_j C_j$ e rappresenta la misura (rispetto all'istante 0) del tempo necessario a completare tutte le attività.
- **Massima lateness Lmax**. Lmax = $\max_j L_j$ e rappresenta il ritardo del job che `e maggiormente in ritardo rispetto alla propria data di consegna. (Si noti che potrebbe anche essere negativa, e in tal caso rappresenta l'anticipo del job che termina con minore anticipo rispetto alla propria data di consegna).
- **Massima tardiness Tmax**. È definita come max (0, Lmax).
- **Tempo di completamento totale**. È pari a $\sum P_j C_j$.

Utilizzando gli elementi analizzati, si può definire una rappresentazione compatta dei problemi di scheduling. Tale notazione `e una notazione a tre campi, nota come notazione di Graham, α/β/γ, ed i tre campi rappresentano rispettivamente:

- **α** il sistema e il numero di macchine,
- **β** le (eventuali) caratteristiche particolari,
- **γ** la funzione obiettivo.

Per quanto concerne il sistema, si indicano le quattro possibili strutture (singola macchina, flow shop, macchine parallele, job shop) con 1, F, P e J rispettivamente. Inoltre, si può indicare il numero di macchine (es. P1, P2, F3, …) e il numero di server per il setup (S1, S2, …).

1.4 Descrizione del problema *P, S1/sj/Cmax.*

Si consideri un problema di schedulazione con macchine parallele identiche e con un server comune (a tutte le macchine) per eseguire operazioni di setup, e lo definiamo formalmente come segue. Sia $J = \{1, \ldots, n\}$ l'insieme dei job da schedulare su m macchine parallele identiche. Ogni $j \in J$ ha un tempo di setup e un tempo di processamento, rispettivamente indicati come s_j e p_j. Prima di eseguire un lavoro j su una macchina, è necessario eseguire un'operazione di setup che richiede s_j unità di tempo da parte di un server comune su tale macchina. Inoltre, sia τ_j la durata del lavoro j, definita da $\tau_j = s_j + p_j$. Il server comune (ad esempio uno strumento, una persona o un gruppo di persone) non può eseguire operazioni di setup mentre la macchina sta eseguendo un lavoro. Pertanto, non devono verificarsi conflitti durante l'esecuzione degli stessi, e le richieste simultanee per il server

comporteranno un tempo di inattività della macchina, essendo che non si può sdoppiare (chiameremo il tempo di inattività di una macchina come *idle time*). Non è consentita la *preemption*.

L'obiettivo è minimizzare il *makespan*[3] (tempo di completamento totale) *Cmax*, ovvero il tempo in cui si riesce a terminare tutte le lavorazioni.

Nella notazione standard, questo problema è indicato come ***P, S1|sj|Cmax.***

Di seguito un esempio:

Si consideri un sistema formato da 2 macchine parallele identiche e un istanza consistente di 4 job, ovvero un *P2, S1|sj|Cmax,* con i tempi caratterizzanti in Tabella 1:

TABELLA 1: DATI PROBLEMA

Tempi lavorazione esempio		
Job	sj	p_j
1	1	5
2	2	2
3	2	3
4	2	2

Nell'immagine (Figura 2) si può vedere la rappresentazione di una delle possibili[4] schedule ottima per il problema di schedulazione, con la rappresentazione temporale dell'utilizzo delle due macchine e del server comune.

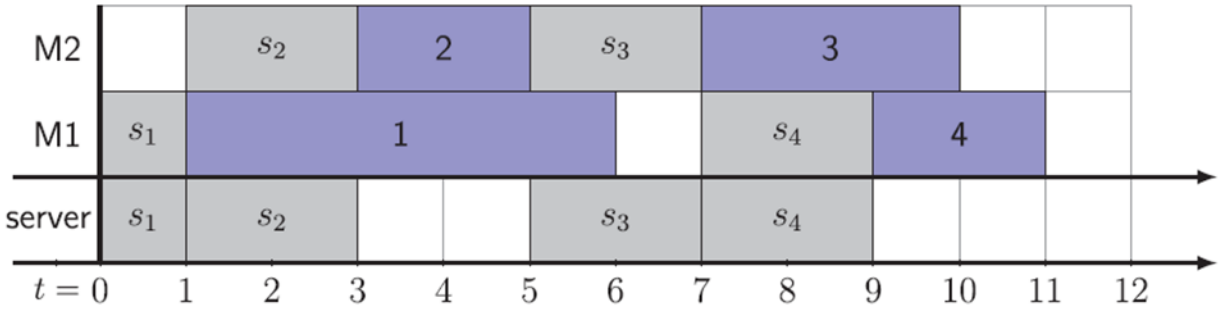


FIGURA 2: GANTT CHART

Si può facilmente dimostrare che t = 11 è il tempo di completamente totale ottimo (per esempio esplicitando tutte le possibili combinazioni).

[3] Si può osservare che minimizzare il tempo di completamento non vuol dire altro che cercare di massimizzare l'utilizzo delle macchine.
[4] La soluzione ottima è una, quella che minimizza il Makespan, ma l'ordinamento dei job nel tempo, ovvero lo schedule, che permette di arrivare alla funzione obbiettivo ottima può non essere univoca.

Capitolo 2 – Modelli e Algoritmi

Dopo una dettagliata introduzione alla Ricerca Operativa, alla programmazione matematica e alla schedulazione è ora possibile addentrarsi nel cuore della trattazione di questo elaborato.

Per prima cosa, verranno mostrati e spiegati i due modelli di programmazione matematica.

Successivamente si faranno due implementazioni che si pongono l'obbiettivo di migliorare le prestazioni dei due modelli, la prima sarà un algoritmo che lavorerà preservando l'ottimalità della soluzione, la seconda, invece, per cercare di migliorare ancora di più le performance del modello andrà a rinunciare alla funzione obiettivo ottima, quindi entrerà nel campo delle euristiche.

Questi modelli sono stati introdotti e studiati da Elidrissi et al. ((Elidrissi A. M., 2018b), (Elidrissi A. R., 2021)).

Altri studi del problema *P, S1|sj|Cmax* sono (Kim M.-Y. a., 2012), (Elidrissi, 2020), (Hamzadayi, 2017) comunque studi di problemi simili si possono trovare in ((João Marcos Pereira Silva, 2023), che è l'articolo di riferimento per questo elaborato.

2.1 Modello 1

Il modello 1 del problema *P, S1|sj|Cmax,* così come il modello 2, sono formulazioni con indice di tempo (detti per questo TIF ovvero Time-Indexed Formulation) che può variare da 0 a T (orizzonte temporale).

Per la prima formulazione si definiscono i setup da eseguire prima di ogni lavorazione come una lavorazione fittizia (dummy job) $J_D = \{n+1, \dots, 2n\}$, e quindi $p_{j+n} = s_j$, ovvero il tempo di processo del job j+n corrisponde al tempo di setup della lavorazione j.

Si utilizzeranno tre set di dati:

- P_j tempi di processo (come abbiamo appena detto $\forall j \in J + J_D$);
- S_j tempi di setup per i job j ($\forall j \in J$);
- T_j tempi di completamento per i job j ($\forall j \in J + J_D$), quindi ci saranno anche i tempi di completamento per i dummy job che saranno uguali a quelli dei job veri e propri, ovvero è come se fosse $T_j + T_j$ con $j \in J$.[5]

Per esempio, ipotizzando 3 job con tempi di processo rispettivamente (1, 2, 3) e tempi di setup rispettivamente (4, 5, 6) avremo l'insieme $P_j = \{1, 2, 3, 4, 5, 6\}$, l'insieme $S_j = \{4, 5, 6\}$,e l'insieme $T_j = \{5, 7, 9, 5, 7, 9\}$.

Inoltre, si indicherà con m il numero di macchine disponibili.

[5] Ricordando la relazione $\tau_j = p_j + s_j$

Ora, si definiscono le variabili binarie[6] y_j^t che indicano che il job j inizia la lavorazione al tempo t su una delle m macchine se assumono valore 1, 0 altrimenti.

FUNZIONE OBIETTIVO:

$$\min C_{max} \tag{1a}$$

VINCOLI:

$$\sum_t^{T-\tau_j} y_j^t = 1 \quad \forall j \in J \tag{1b}$$

$$\sum_{j\in J} \sum_{s=\max\{0,\ t-\tau_j+1\}}^{t} y_j^s \leq m \qquad t = 0, \dots, T-1 \tag{1c}$$

$$\sum_{t=0}^{T-s_j} y_j^t = 1 \quad \forall j \in J_D \tag{1d}$$

$$\sum_{j\in J_d} \sum_{s=\max\{0,\ t-s_j+1\}}^{t} y_j^s \leq 1 \qquad t = 0, \dots, T-1 \tag{1e}$$

$$C_{max} \geq \sum_{t=0}^{T-\tau_j} (t + \tau_j)\, y_j^t \quad \forall j \in J \tag{1f}$$

$$y_j^t = y_{j+n}^t \quad \forall j \in J;\ \text{t}= 0, \dots, T-\tau_j \tag{1g}$$

$$y_j^t \in \{0,\ 1\} \quad \forall j \in J \cup J_D;\ \text{t}= 0, \dots, T-\tau_j \tag{1h}$$

- La funzione obiettivo (1a) minimizza il makespan;
- I vincoli (1b) garantiscono che ogni lavoro debba essere svolto esattamente una volta;
- I vincoli (1c) impongono che al massimo m job possano essere lavorati in qualsiasi momento t. (quindi che venga rispettano il numero di macchine disponibili);
- I vincoli (1d) indicano che ogni dummy job deve iniziare a una determinata ora t;
- I vincoli (1e) garantiscono che, in qualsiasi momento dato, al massimo un dummy job possa essere elaborato;
- I vincoli (1f) calcolano e definiscono il makespan;
- I vincoli (1g) assicurano che il job j e il dummy job j+n debbano iniziare contemporaneamente;
- Infine, i vincoli (1h) definiscono la natura binaria delle variabili decisionali.

Si può notare che sono state definite le variabili binarie con t= $0, \dots, T-\tau_j$, facendo così si è sicuri che l'ultimo job in lavorazione finisca al massimo a T (sarà così anche per il modello 2).

Un lettore attento, ora, può domandarsi, *ma T a cosa lo si fa corrispondere per essere sicuri che sia abbastanza grande da permettere al modello di trovare correttamente la soluzione ottima?*[7]

Una soluzione può essere quella di definire T come la somma di tutti i tempi di completamento, che equivale a dire che tutte le lavorazioni vengono svolte su una sola macchina, il che ci fa essere sicuri che la soluzione ottima, e qualsiasi altra soluzione ammissibile, sia inferiore del T massimo dato.

[6] Le variabili binarie possono assumere solo due valori, 0 e 1.
[7] Se T fosse troppo piccolo chiaramente in questo arco temporale il solver non riuscirebbe a schedulare tutte le lavorazioni rispettando i vincoli, ovvero il modello risulterebbe infattibile.

Per questo modello e per il secondo useremo $T = \sum \tau_j$, però, ovviamente, ci sono strade migliori, che verranno trattate in seguito in questo elaborato.

Si può osservare che questo modello crea tantissime variabili e, in generale, più variabili contiene un modello, più per un solver è complicato risolverlo. Stesso discorso per i vincoli.

Quindi la domanda sorge spontanea: *si può fare meglio?* La risposta è sì.

2.2 Modello 2

Come semplificare la formulazione del primo modello?

I vincoli (1g) stabiliscono un collegamento diretto tra i job e i dummy job; quindi, nei vincoli (1e) le variabili inerenti ai dummy job possono essere sostituite da quelle dei job reali. Pertanto, l'insieme J_D non serve più, così come i vincoli (1d) ed (1g).

L'insiemi di dati allora sarà formato da:

- P_j tempi di processo per i job j;
- S_j tempi di setup per i job j;
- T_j tempi di completamento per i job j.

Se si usa l'esempio di prima ipotizzando 3 job con tempi di processo rispettivamente (1, 2, 3) e tempi di setup rispettivamente (4, 5, 6) si avrà l'insieme $P_j = \{1, 2, 3\}$, l'insieme $S_j = \{4, 5, 6\}$,e l'insieme $T_j = \{5, 7, 9\}$.

A questo punto, si introducono le variabili intere w^t che indicano il numero di macchine inattive nel time bucket t (quindi potrà assumere tutti i valori da 0 a m, numero di macchine disponibili). Il solver, ovviamente, vorrà tenere queste variabili il più possibile vicine allo 0, ovvero vorrà minimizzare per ogni time bucket l'idle time (tempo inattivo) sulle macchine, che vuol dire cercare di massimizzare l'utilizzo o minimizzare il tempo di completamento.

Inoltre, facendo qualche aggiustamento nelle sommatorie dei vincoli (1c) e (1e), si può finalmente formulare il modello 2 come segue:

FUNZIONE OBIETTIVO:

$$\min C_{max} \tag{2a}$$

VINCOLI:

$$\sum_{t}^{T-\tau_j} y_j^t = 1 \quad \forall j \in J \tag{1b}$$

$$C_{max} \geq \sum_{t=0}^{T-\tau_j} (t + \tau_j)\, y_j^t \quad \forall j \in J \tag{1f}$$

$$\sum_{j \in J} \sum_{s=\max\{0,\ t-\tau_j+1\}}^{\min(t, T-\tau_j)} y_j^s + w^t = m \quad t = 0, \dots, T-1 \tag{2b}$$

$$\sum_{j \in J} \sum_{s=\max\{0,\ t-s_j+1\}}^{\min(t, T-\tau_j)} y_j^s \leq 1 \quad t = 0, \dots, T-1 \tag{2c}$$

$$y_j^t \in \{0, 1\} \quad \forall j \in J; \quad t = 0, \dots, T-\tau_j \tag{2d}$$

$$w^t \in \{0, \ldots, m\}; \ t = 0, \ldots, T-1 \tag{2e}$$

- I vincoli (2b) e (2c) sono l'aggiornamento dei vincoli (1c) e (1e) rispettivamente (adattati alla nuova definizione delle variabili); i primi garantiscono che non venga sforato il numero di macchine, i secondi garantiscono che il server per il setup venga usato al più su una macchina in ogni time bucket;
- I vincoli (1b) e (1f) sono esattamente uguali al modello 1, quindi rispettivamente garantiscono che ogni job venga svolto esattamente 1 volta e definiscono il makespan;
- I vincoli (2d) e (2e) sono la definizione delle variabili.

	Numero di variabili			
	Binarie	Intere		
Modello	y	w	Cmax	Numero di vincoli
1	2nT	0	1	3n+nT+2T
2	nT	T	1	2n+2T

TABELLA 2: NUMERO VARIABILI E NUMERO VINCOLI PER I DUE MODELLI

Valgono le considerazioni fatte per il modello 1 riguardo alla definizione delle variabili binarie e sul tempo T massimo da fornire al modello.

Si può osservare nella Tabella 2 che questa formulazione è più "snella" della precedente, in particolare, di due categorie di vincoli in meno e, soprattutto, nel numero di variabili totali create, che sono state quasi dimezzate.

2.3 Algoritmo per determinazione tempo T

T definito come somma dei tempi di completamento è evidentemente un modo poco efficiente di determinare l'orizzonte temporale per questo problema di schedulazione[8], perché definendo le variabili proporzionalmente rispetto al tempo, allora, più la stima dell'orizzonte è migliore, più leggero sarà il modello, e, teoricamente, più performante.

L'obbiettivo è quindi quello di trovare un T da dare all'algoritmo sufficientemente grande da essere sicuri che la soluzione ottima rientri nell'orizzonte temporale e abbastanza piccolo per evitare la creazione di inutili variabili.

In questa sezione viene proposto un semplice algoritmo che, in maniera rapida, fornisca un orizzonte temporale ragionevole al modello da risolvere. Di seguito l'algoritmo:

START
//parte 1: inizializzazione
T = 0
For num_macchine (**indice** i, **passo** 1):
    $Tm_i = 0$
//parte 2: calcolo Tm_i

[8] Ricordando che equivale al tempo necessario a svolgere tutti i job su una sola macchina.

```
For arr_eccesso (numero_job/num_macchine) (passo 1):
    For num_macchine (indice c, passo 1):
        if Contatore_job >= J:
            VAI A PARTE 4
        Else:
            Tm_c = Tm_c + tempo_setup_contatore_job
            //parte 3: controllo vincolo unica risorsa setup
            For num_macchine (indice s; passo 1):
                if s = c:
                    VAI A PARTE 4
                Elif Tm_s >= Tm_c:
                    VAI A PARTE 4
                Else:
                    Tm_s = Tm_c
            //parte 4: aggiornamenti
            Tm_c = Tm_c + tempo_processo_contatore_job
        Contatore_job = Contatore_job + 1
//parte 5: definizione orizzonte T
T = massimo (Tm_i)
//parte 6: esecuzione modello
END
```

Il funzionamento dell'algoritmo consiste nell'assegnare in ordine ad ogni macchina, un job (nell'ordine in cui ci vengono dati in input, quindi job 1 eseguito da macchina 1, job 2 eseguito da macchina 2, job 3 eseguito da macchina 3 se le macchine sono più di 2, altrimenti eseguito dalla macchina 1 e così via…), finché non finiscono. Tenendo conto del tempo che ogni macchina impiega ad eseguire tutte le lavorazioni che le sono state assegnate, rispettando il vincolo che solo in una macchina alla volta si può eseguire il setup, l'orizzonte T sarà determinato dalla macchina che impiega più tempo, ovvero T sarà pari al massimo tra i tempi macchina Tm.

Nella parte 1 si inizializzano le variabili necessarie per l'esecuzione dell'algoritmo: ovviamente il tempo T, un contatore che serve per il coretto funzionamento dell'algoritmo, e, per ogni macchina, il tempo macchina Tm.

Nella parte 2 si inizia l'implementazione delle variabili. Si crea un ciclo da ripetere j/m volte (numero di job diviso numero di macchine), ovvero le volte necessarie ad assegnare ogni job ordinatamente alle macchine. Successivamente si arrotonda per eccesso questo rapporto per rendere l'algoritmo adatto ad ogni combinazione di numero di job e numero di macchine (potrebbero non essere multipli, quindi il rapporto non intero e, chiaramente, non ha senso ripetere delle iterazioni un numero di volte non intero). I due contatori servono a tenere il conto dei job che sono stati assegnati. Il valore del contatore rappresenta l'i-esima lavorazione.

Ora, viene creato un nuovo ciclo, da ripetere ogni volta per ogni macchina (indice c, quindi se 2 macchine verrà ripetuto due volte e c assumerà valori 1 e 2). Avendo arrotondato il rapporto $|J|$/m in precedenza, è come se in tutti i casi in cui i due fattori del rapporto non sono multipli, si stesse aumentando il numero di job da assegnare, quindi si deve controllare che il job che si sta per allocare

esista veramente, se è così si può procedere con la sua assegnazione alla c-esima macchina, se non è così vuol dire che si è nel caso di arrotondamento e quindi ci si bisogna fermarsi, perché si è già assegnato tutte le lavorazioni alle macchine (essendo che si allocano i job ordinatamente).

A questo punto, il tempo di lavorazione totale della macchina c-esima sarà maggiorato del tempo di completamento del job appena assegnato (tempo di setup e tempo di processo, che verrà addizionato in due slot separati il tempo di processo lo si aggiungerà nella parte 4).

Arrivati alla parte 3, la parte più delicata dell'algoritmo, si deve verificare che il setup venga eseguito su una macchina alla volta. Un modo furbo per evitare che più macchine eseguano il setup contemporaneamente è portare tutti i tempi delle altre macchine uguali al tempo della c-esima macchina più il tempo di setup del job in lavorazione (ovviamente se sono minori, se sono maggiori non c'è pericolo che il setup possa essere contemporaneo), così nessuna macchina può iniziare il setup di un suo job prima che la c-esima non abbia finito il suo. Lo si ottiene confrontando i tempi della c-esima macchina con i tempi delle altre m-c macchine prese singolarmente (s-esima).

Si giunge alla parte 4, dove viene aggiunto il tempo di processo del job alla macchina c-esima e infine si aggiorna il contatore (ovvero possiamo al job successivo).

Ora non resta altro che prendere il massimo tra i tempi delle macchine Tm, ovvero il T, ed eseguire il modello (parte 5 e 6).

Un'ultima osservazione che si può fare è che il T dato in input al modello è una soluzione ammissibile del problema.

2.4 Algoritmo di assegnazione

In questa sezione verrà introdotto un algoritmo, che ha l'obbiettivo di migliorare ancora di più le prestazioni dei due modelli, di alleggerire il lavoro del solver. Lo si fa con un'euristica, quindi si accetta di scostarsi dalla soluzione ottima (di un massimo del 10% - 20%) in cambio di un risparmio in termini di calcolo.

Quello che viene proposto in questo elaborato è un algoritmo che lavori in sinergia con il modello, non che lo sostituisca per la ricerca della soluzione ottima, ovvero un algoritmo di assegnazione che in base alla combinazione del numero di macchine e del numero di job (vedi Capitolo 5) andrà ad assegnare ad un certo numero di variabili binarie il valore 1, ovvero allocherà alle macchine alcuni job di default. Fatto questo lascerà lavorare il modello per trovare la soluzione ottima nel sottoproblema rimanente.

Di seguito l'algoritmo:

START
//Parte 1: determinazione orizzonte T
Si esegua l'algoritmo sopra descritto (2.3 Algoritmo per determinazione tempo T)
//Parte 2: ordinazione dati di input
Ordinamento job per s_j crescenti e, a parità di s_j, per p_j decrescenti.
//Parte 3: Inizializzazione

Arrotondamento = X (Si veda capitolo 3).
Num_var_assegnare = **arr_eccesso** (arrotondamento * numero_job)
Job_assegnato = 0
Tempo_server = 0
For num_macchine (**indice** i, **passo** 1):
    $Tm_i = 0$
//Parte 4: assegnazione e aggiornamenti
For arr_eccesso (Num_var_assegnare /num_macchine) (**passo** 1):
    For macchine (**indice** m, **passo** 1):
        if Job_assegnato >= Num_var_assegnare:
            VAI A PARTE 5
        Else:
            y[Job_assegnato] [massimo (Tempo_server, Tm_m)] = 1
            Tm_m = **massimo** (Tempo_server, Tm_m]) + tempo_setup $_{Job_assegnato}$
            Tempo_server = Tm_m
            $Tm_m = Tm_m$ + tempo_ processo $_{Job_assegnato}$
            Job_assegnato = Job_assegnato + 1
        Tempo_server = **massimo** (Tempo_server, **minimo** (tra tutti i Tm))
//Parte 5: esecuzione modello
Si esegua il modello 1 o 2[9]
If output modello = NOT_SOLUTION_FOUND:
    $C_{max} = T$
END

Il concetto di base di questa semplice euristica è assegnare ordinatamente alle m macchine del problema un numero di job, non in ordine di input come nel precedente algoritmo, ma ordinandoli per tempi di setup crescenti e, a parità di tempo di setup, per tempi di processo decrescenti. L'obbiettivo dell'algoritmo è di assegnare alle macchine all'inizio i job con tempo di setup minore, ma tempo di processo maggiore possibile. Questo per due ragioni: la prima è che allocando alle macchine le lavorazioni che richiedono per poco tempo il server comune consente, al tempo iniziale, di rendere minima l'attesa del server per le altre macchine. Un esempio può essere quello di avere due macchine, a tempo 0 si assegna alla macchina 1 una lavorazione con un tempo di setup pari a 10. Vuol dire che la macchina 2 dovrà aspettare fino a t=10 per poter iniziare il suo primo job (da 0 a 10 sarà idle perché ovviamente non sta ancora processando nessun job). Se alla macchina 1 invece si assegna un job con tempo di setup pari a 2, per esempio, la macchina 2 aspetterà solo t=2 per poter iniziare la sua lavorazione, rimanendo idle solo per due unità di tempo rispetto alle 10 di prima. Questo concetto si può estendere anche a più macchine e l'effetto si moltiplica. La seconda ragione è che l'algoritmo cerca di mantenere tra le m macchine uno scostamento reciproco abbastanza grande per evitare che delle macchine debbano dover aspettare il server. Ovvero vuole che le macchine, inizialmente, siano sbilanciate, che finiscano i rispettivi job a tempi il più possibile lontani tra loro. Questo si ottiene allocando i job con tempo di processo più grosso possibile e tempo di setup più piccolo possibile in modo che il gap non debba essere troppo grande. Poi, ovviamente, la soluzione

[9] In realtà il modello è inizializzato prima della parte 3 dell'algoritmo per potere creare tutte le variabili e, quindi, per potere assegnare alcune di esse ad 1.

ottima che minimizzi il tempo di completamento la si ottiene bilanciando il più possibile i tempi tra le varie macchine, ma a questo ci penserà il solver. L'obbiettivo di questo algoritmo è di assegnare i job alle macchine riducendo al minimo gli idle time.

Inoltre, essendo che, come abbiamo detto prima, il T fornito al modello dell'algoritmo precedente è una soluzione ammissibile del problema, in caso in cui il modello non riesca a trovare neanche una soluzione nei limiti di tempo dati (Si veda: capitolo 3) fornirà in output una soluzione ammissibile esattamente pari a T.

Entrando più nel dettaglio; nella parte 1 si richiama l'algoritmo per la determinazione dell'orizzonte T e nella parte 2 si ordinano i job. Con la parte 3 si inizializza l'algoritmo. In primo luogo, si determina il numero di variabili (Num_var_assegnare) che l'algoritmo dovrà mettere ad 1 (la percentuale X di aggiornamento gliela daremo noi in input), in caso di numero decimale si arrotonderà per eccesso. Poi si definisce il contatore Job assegnato per tenere conto del numero totale di job che sono stati assegnati e per indicare il job che andrà assegnato successivamente, inoltre, si definiscono le variabili per i tempi, del server e delle macchine.

Nella parte 4 si entra nel cuore dell'euristica. Viene creato un ciclo da ripetere Num_var_assegnare /num_macchine, ovvero il numero di volte necessario ad assegnare tutti i job che sono stati prefissati alle macchine. Anche qua si arrotonda per eccesso per le stesse considerazioni che sono state discusse precedentemente nell'elaborato.

A questo punto, viene creato un nuovo ciclo, da ripetere per ogni macchina. All'inizio si verificherà che non si è arrivati al numero massimo di job da assegnare. Se così non è, si assegna il job alla macchina e poniamo ad 1 la variabile binaria in questione (y con indice j= Job_assegnato e indice T= massimo tra il tempo server e il tempo della macchina in questione[10]).

Fatto questo, vengono aggiornati i tempi del server e delle macchine. Il tempo della macchina sarà il massimo tra il tempo server e il tempo della macchina stesso a cui va aggiunto il tempo di setup del job (per avere la sicurezza che il job non inizi prima che il server sia disponibile). Il tempo del server sarà uguale al tempo della macchina appena aggiornato. Ora si aggiunge il tempo di processo del job al Tm della macchina, e poi, si aggiorna il contatore dei job, ovvero il job successivo entra in lavorazione.

Eseguito il ciclo su tutte le macchine, viene riaggiornato il tempo del server come il massimo tra il tempo del server stesso e il minimo tra i Tm delle varie macchine.

Infine il modello viene eseguito (parte 5). Se il modello nei limiti di tempo dati (Si veda: capitolo 3) non riesce a trovare una soluzione ammissibile darà come tempo di completamento in output il T determinato dall'algoritmo, che, si ricorda, è una soluzione ammissibile.

[10] Per essere sicuri che il job non inizi prima che il server sia disponibile.

Capitolo 3 - Analisi dei risultati computazionali

3.1 Istanze e dati in input

In questa sezione verranno valutati i risultati ottenuti dall'esecuzione delle sei diverse varianti (modello 1, modello 2, modello 1 con algoritmo per tempo, modello 2 con algoritmo per tempo, modello 1 con euristica di assegnazione e modello 2 con euristica di assegnazione).

Queste sei combinazioni sono state implementante con il linguaggio di programmazione Python (i codici Python sono disponibili in Appendice A), con l'ausilio della libreria Python-MIP, una libreria pensata apposta per la modellizzazione di MIP.

Il modello, una volta ultimato e una volta fornitogli i dati in input per la schedulazione (istanza) da analizzare, verrà risolto dal solver Gurobi entro un certo limite di tempo (si veda sotto).

Le istanze in input sono state generate utilizzando il metodo descritto da (Kim M.-Y. a., 2012). In particolare, le istanze sono state create casualmente dalla combinazione di quattro parametri (n, m, α, ρ); n è il numero di job (che assumerà valori pari a n = 10, 20, 30, 50, 75 e 100), m il numero di macchine (che assumerà valori pari a m = 2, 4, 6, 10), α il grado di diversità tra i tempi di setup e i tempi di processo (che assumerà valori pari a 0.1, 0.3, 0.5). Infine, ρ si può definirlo come fattore di gravità del tempo di setup, in poche parole quanto è lungo il tempo di setup (per approfondimenti sul tema (Silva, 2021)). Per ρ ci saranno tre livelli "di gravità" (dato un set di tempi di processo, questo assumerà tre possibili livelli diversi di tempi di setup).

Si è testato in totale 745 istanze. Per $\alpha = 0.1$ si sono testate cinque istanze per combinazione di n, m, ρ. Per gli α successivi (0.3, 0.5) si è tralasciato ρ (utilizzando per semplicità solo il livello 1), testando 5 istanze per tutte le combinazioni di m e n.

Tutte queste istanze di input sono disponibili online al sito:
https://github.com/JoaoMarkcos/Benchmarks/tree/main/PMSPwCS.

Al solver per ogni valore di n è stato dato un limite di tempo per poter trovare la soluzione ottima (Tabella 3).

TABELLA 3: TEMPI DI CALCOLO MASSIMI DEL SOLVER IMPOSTATI

Limite di tempo	
n	tempo massimo
10	60 s
20	60 s
30	100 s
50	200 s
75	300 s
100	300 s

Inoltre, come anticipato nella descrizione dell'algoritmo di assegnazione, per ogni combinazione di m ed n è stata data in input una percentuale di job da allocare alle macchine nell'euristica (Tabella 4).

TABELLA 4: ASSEGNAZIONE

Percentuale assegnazione			
Job	Macchine	Percentuale	Numero variabili
10	2	0.1	1
10	4	0.1	1
10	6	0.1	1
20	2	0.1	2
20	4	0.1	2
20	6	0.1	2
20	10	0.1	2
30	2	0.15	5
30	4	0.15	5
30	6	0.1	3
30	10	0.1	3
50	2	0.3	15
50	4	0.3	15
50	6	0.2	10
50	10	0.2	10
75	2	0.4	30
75	4	0.4	30
75	6	0.3	23
75	10	0.3	23
100	2	0.5	50
100	4	0.5	50
100	6	0.4	40
100	10	0.4	40

Le percentuali sono state decise contrapponendo da un lato il tenerle più basse possibile per diminuire lo scostamento eventuale delle soluzioni ottime con quelle del modello esatto (considereremo accettabile uno scostamento non superiore al 10% - 20%) e dall'altro il tenerle il più alte possibile per migliorare le prestazioni a livello di tempo del solver. Quindi altre scelte sul numero di variabili da far assegnare all'algoritmo porteranno a risultati anche molto diversi tra loro. Aumentando le percentuali diminuirà il tempo di calcolo e peggiorerà lo scostamento, viceversa abbassandole.

Il solver può dare in output tre diversi scenari riguardo lo stato della funzione obiettivo: ottima, ovvero ha trovato la soluzione ottima, ammissibile, ovvero ha trovato una delle possibili soluzioni

ammissibili (nel caso che non riesca a trovare la soluzione ottima nel tempo datogli) o che non ha trovato soluzioni (vuol dire che nel limite di tempo non è riuscito neanche a trovare una soluzione ammissibile). In realtà, il solver potrebbe dare in output anche un altro stato della funzione obiettivo: inammissibile. Questo vorrebbe dire che il modello/algoritmo datogli non rispetta almeno un vincolo, ovvero, ci sarebbe un errore nella formulazione. Quindi, ovviamente, non si otterranno mai come output ai test sottostanti questo stato della soluzione, altrimenti gli algoritmi sopra descritti sarebbero sbagliati.

Oltre allo stato della soluzione, il solver darà in output anche il tempo impiegato per trovare la soluzione (in caso di soluzione non trovata o ammissibile sarà uguale al tempo massimo datogli), il valore vero e proprio della soluzione (la funzione obiettivo, il C_{max}) e il migliore bound, che si può intendere, facendo qualche semplificazione, a quanto il solver è arrivato vicino alla soluzione ottima dell'istanza. Ovvero, più il bound e la funzione obiettivo sono distanti, più il solver è distante dalla soluzione[11]. Se il solver ha trovato la soluzione ottima, la funzione obiettivo e il bound coincideranno.

Un'ultima osservazione che si può fare prima di commentare i risultati è che il modello 1 e il modello 1 eseguito con l'algoritmo per l'orizzonte temporale avranno un tempo T trovato e dato in input al solver che sarà il doppio rispetto ai corrispettivi del modello 2, questo a causa della definizione dei dummy job. Nell'algoritmo di assegnazione, per migliorare ulteriormente le prestazioni del modello 1, questo "difetto" è stato coretto, quindi, i T dei due modelli coincideranno.

I dati ricevuti in output dal solver per ogni istanza di input sono stati poi aggregarti nelle tabelle di seguito (tutti i file con i risultati di ogni istanza testata si possono trovare a questo link: https://github.com/SimoneM02/Tesi-).

3.2 Risultati e commenti

Si può partire con l'analizzare le istanze con $\alpha = 1$. La Tabella 10 e la Tabella 11 mostrano l'andamento delle sei versioni con 2 macchine, per differenti livelli di ρ. Si nota che i due modelli base riescono a trovare soluzioni ottime fino a 30 job (anche se con 30 ne trovano rispettivamente 6 e 5 su 15). Quindi "performano bene" fino a 20 lavorazioni. Riguardo alle implementazioni del modello 1, si vede che con un orizzonte temporale più corto (riducendo le variabili) il solver riesce a trovare sette soluzioni ottime in più, riducendo di più del 20% i tempi di calcolo. Tuttavia anch'esso dopo 30 job non riesce a trovare soluzioni ottime. Inoltre, si vede che il solver, implementando il modello con l'algoritmo per il tempo, fa fatica in molte istanze a trovare una soluzione ammissibile. Questo problema si verificherà anche con $m > 2$ e sia per il modello 1 sia per il modello 2. Questo perché il solver decide di prendere strade diverse per risolvere il problema dal modello base rispetto al modello con T e in alcuni casi può risultare più difficile. L'algoritmo di assegnazione riesce a trovare 53 ottimi (si intende il numero di ottimi trovati relativi al problema semplificato), di cui 35 coincidenti a quelli del modello base (riesce a trovare qualche soluzione ottima anche con 50 job). Questo introduce un altro miglioramento in termini di tempo. Guardando complessivamente

[11] Essendo una funzione obiettivo di minimo il bound sarà sempre minore o uguale della soluzione ottima, sarà allora un Lower bound. Per approfondimenti sull'argomento:
https://www.math.unipd.it/~luigi/courses/ricop0809/ro_13.m04.BeB.01.pdf

l'andamento del tempo si nota che come "migliorando" il modello con i due algoritmi, il tempo di calcolo mediamente scende, fino a 50-75 job dove le tre versioni iniziano a convergere al massimo tempo datogli, essendo che sia il modello 1, sia il modello 1 + T, da 50 job in poi non riescono a trovare soluzioni ottime, e, l'algoritmo di assegnazione, da 75 job in poi. Tuttavia, è interessante osservare l'andamento delle F.O.; per i due algoritmi l'andamento è molto simile, mentre il modello base "esplode" a 50 job, ovvero ha delle F.O. altissime. Questo vuol dire che da 50 job in poi il solver con questo modello, nel tempo datogli, riesce a malapena a trovare una soluzione ammissibile, mentre con i due algoritmi è molto più vicino al trovare soluzioni ottime (infatti in molti casi il bound, da 50 job in su del modello base rimane a 0, in pratica è come se non avesse ancora iniziato a risolvere il problema, mentre per i due algoritmi le F.O. e i bound sono molto più vicini tra di loro, anche se abbastanza distanti). Se dessimo tempo infinito alle tre versioni, i modelli con gli algoritmi ci metterebbero molto meno tempo rispetto al modello base a trovare delle soluzioni ammissibili, e probabilmente anche gli ottimi, coll'euristica che primeggerebbe.

L'incremento del numero di ottimi trovati dall'euristica di assegnazione (quindi ottimi del problema semplificato, non per forza coincidenti con gli ottimi del modello base) rispetto al modello base e all'algoritmo con T è dato anche dal fatto che il solver deve risolvere un problema più semplice, dove in molti casi la F.O. è maggiore rispetto al problema di partenza. Comunque, analizzando gli scostamenti, si vede che assegnando di default alcune variabili, la soluzione peggiora solo leggermente (gli scostamenti medi e massimi sono molto piccoli rispetto al valore delle F.O ottime).

Passando al modello 2, si nota un andamento molto simile delle tre versioni rispetto al modello 1, con, però, un crollo di ottimi trovati dall'algoritmo T (da 42 a 28), a causa di un netto incremento di soluzioni non trovate. Come per il modello 1, gli andamenti dei tempi e delle F.O. nei tre scenari è molto simile, con i tempi che hanno un andamento quasi uguale tra loro (leggermente più bassi i due algoritmi) e con il modello base che esplode verso i 50 job.

Guardando la Tabella 5, però si osserva che il modello 2 base esplode (ovvero t e FO tendono rispettivamente al limite di tempo e alla FO massima, ovvero la somma dei tempi di completamento) molto meno rispetto al modello 1 base, questo grazie al dimezzamento delle variabili e alla riduzione dei vincoli. Le due versioni delle euristiche di assegnazione forniscono le stesse prestazioni (sarà così per tutte le m ed è l'andamento che si prevedeva). Anche per l'algoritmo per la determinazione di T (sia per il modello 1, sia per il modello 2) le differenze sono molto piccole, anche se più marcate rispetto all'assegnazione, questo perché nell'assegnazione è stato coretto il calcolo di T, escludendo i dummy job dal conteggio. Si nota che le somme dei tempi e delle F.O. sono più piccole negli algoritmi per T rispetto agli algoritmi di assegnazione, questo perché nelle somme non sono state considerate le soluzioni non trovate, quindi è un confronto fuorviante (i tempi medi invece tornano coerenti).

Questi andamenti si ripeteranno, anche se sempre più in maniera smorzata, anche con m > 2.

Passando a m = 4 (Tabella 12 e Tabella 13) si osserva per entrambi i modelli un netto miglioramento delle prestazioni dell'algoritmo di assegnazione che riesce a trovare 72 e 73 ottimi su 90 rispettivamente con il modello 1 e il modello 2, anche se peggiorano molto gli scostamenti dalle soluzioni "reali" rispetto a m=2. Il picco di soluzioni ottime lo si ha con livello 1, dove l'algoritmo riesce a trovare 30/30 soluzioni ottime (anche in m=2 il picco lo si registra al livello 1). Per quanto riguarda i tempi, gli algoritmi per T e i modelli base sono molto vicini tra loro, mentre si evince un

netto decremento dei tempi per l'euristica (essendo che al livello 1 trova in tutti e due i modelli tutte le soluzioni ottime, il tempo crolla molto). Tuttavia, per ρ = 2 e 3 l'algoritmo fa molta più fatica a trovare gli ottimi per istanze grandi (addirittura per 75 e 100 job trova 5 ottimi al livello 1 e 0 al livello 3). Ancora, l'andamento delle F.O. è molto simile al caso m=2 con gli algoritmi "vicini" e i modelli base che esplodono, questa volta intorno ai 75 job. Infine, i modelli base e i modelli con l'algoritmo per la determinazione di T hanno un numero di ottimi praticamente identico (45 e 46 per il modello 1, 48 e 49 per il modello 2). Ancora globalmente (Tabella 6) si vede la grande differenze di prestazioni tra i due modelli base e le somiglianze reciproche degli algoritmi.

Le Tabella 14, Tabella 15 e Tabella 7 analizzano le istanze con 6 macchine. Gli andamenti medi dei tempi e delle funzioni obiettivo sono simili al caso m=4, con l'esplosione del modello base sempre più avanti, ora a 100 job (livello 1) per il modello 2 e a 75 (livello 3) per il primo modello. Gli algoritmi di assegnazione riescono a migliorare il loro numero di ottimi (75 per il modello 1, 76 per il 2). Si nota che il miglioramento avuto è per il livello ρ = 2, dove anche qua abbiamo 30/30 ottimi trovati in entrambi i casi, mentre per il terzo livello l'euristica fatica ancora molto. Gli scostamenti restano sempre dello stesso ordine di grandezza, anche se quasi tutte le istanze sono affette da un minimo peggioramento rispetto all'ottimo assoluto. I modelli base si migliorano entrambi a numero ottimi trovati, da 46 a 50 per il modello 1, da 48 a 54 per il modello 2. Entrambi "faticano" ancora con le istanze grandi (50, 75 e 100 lavorazioni). Invece i modelli con l'algoritmo per il tempo peggiorano entrambi il numero di ottimi trovati. Gli algoritmi di assegnazione si differenziano sempre di più dagli algoritmi di determinazione T a livello di tempi di calcolo, grazie alla grande differenza di ottimi trovati.

A livello globale si vedono ancora le difficoltà in termini di F.O. delle versioni base dei modelli.

Per ultime si considerano le Tabella 16, Tabella 17 e Tabella 8, relative a 10 macchine. Qua è dove tutte le versioni hanno le prestazioni migliori. Per quando concerne il modello 1 base, si osserva un miglioramento netto per il livello 1 di tempi di setup, per quanto riguarda le F.O. (molto simile alle versioni successive con gli algoritmi), ma per i livelli 2 e 3 torna alle distanze osservate in precedenza. Migliora molto anche il numero di ottimi, considerando che sono state testate 15 istanze in meno (quelle relative a 10 job). Ancora una volta il modello con l'algoritmo per T non introduce miglioramenti a livello di numero di soluzioni ottime trovate, ma solo in termini di F.O. medie. L'algoritmo di assegnazione questa volta riesce a trovare ottimi (71 su 75 gli ottimi totali) anche per il terzo livello. Gli scostamenti restano bassi ma su tutte le istanze. L'andamento del modello 2 invece, a livello di F.O., si allinea con le due versioni successive, non mostrando più l'esplosione classica del modello base. Tuttavia, il numero di ottimi resta circa uguale a quello trovato dal modello 1 base (52 e 50). Le versioni successive si comportano ugualmente nei due modelli.

Per tutti le sei versioni, per tutte le m, all'aumentare del livello ρ, aumentano i tempi di calcolo, le FO e diminuiscono gli ottimi trovati. Questo anche banalmente perché aumentando i tempi di setup, aumentano l'orizzonte temporale e quindi, aumentano le variabili in gioco.

Riassumendo, con α = 1, si hanno le versioni base che per istanze grandi (>=50 job) al diminuire delle macchine risultano completamente inefficienti, con il modello 1 molto peggiore del modello 2 a causa della presenza dei dummy job che ne raddoppiano variabili e vincoli. L'algoritmo per la determinazione del T introduce miglioramenti solo nelle F.O. medie, arriva molto più vicino alle soluzioni ottime rispetto ai modelli base, ma, a numeri di ottimi e a tempi non introduce

miglioramenti, anzi, in molti casi peggiorano le prestazioni globali. Le F.O. sono molto vicine a quelle dell'algoritmo di assegnazione, quest'ultimo con il vantaggio di avere delle lavorazioni assegnate in partenza riesce a migliorare il numero di ottimi e i tempi di calcolo computazionale (tuttavia, per numero di macchine piccole e grandi istanze anche loro non trovano soluzioni ottime nel tempo). Questo porta a un lieve peggioramento delle funzioni obiettivo rispetto a quelle del problema di partenza, ma gli scostamenti sono minimi[12]. A parità di tempo di processo, aumentando il tempo di setup, si ha un peggioramento nelle prestazioni di tutte le sei versioni. Questo si avrà anche per gli α successivi, quindi si considererà solo ρ = 1 per lo studio delle prossime istanze.

Per lo studio di α = 0,3 e 0,5 oltre a studiare solo il primo livello di tempi di setup, si escluderà dalle tabelle l'algoritmo di determinazione dell'orizzonte temporale, perché si è visto che non porta a miglioramenti consistenti. Inoltre, aumentando α, le F.O. trovate (le istanze sono state generate) tendono ad allontanarsi dall'euristica di assegnazione, convergendo verso i livelli del modello base. Quindi si può considerare l'andamento dell'algoritmo di determinazione di t per α > 0,1 circa uguale al modello base.

Guardando la Tabella 18: p = 1, alfa = 0.3) si osserva che i due modelli base trovano lo stesso numero di ottimi (47 il modello 1, 46 il modello 2). Entrambi dopo 20 lavorazioni iniziano a non trovare ottimi per numeri di macchine piccole, mentre per m = 6 riescono ancora a trovare ottimi (5 e 4 rispettivamente), per 30 job e per m =10 trovano ottimi per 30 e 50 lavorazioni. In entrambi i modelli si ha una netta crescita delle somme delle F.O. per numeri di macchine piccoli (sempre molto maggiori quelli del modello 1). Per quanto riguarda i due algoritmi di assegnazione, rispetto al caso α = 0,1 si accentuano le differenze tra modello 1 e modello due, i quali trovano rispettivamente 59 e 64 soluzioni ottime, di cui 36 e 41 uguali ai problemi reali. Questa differenza porta anche a una differenza in termini di tempo tra i due modelli, anche se minima. Per quanto riguarda le funzioni obiettivo, queste sono molto simili tra i due algoritmi, con addirittura l'euristica del modello 1 che ha un totale delle F.O. minori rispetto a quelle del modello 2, anche se riesce a trovare meno ottimi. Anche i due algoritmi hanno dei "picchi" in corrispondenza delle lavorazioni con 2 macchine parallele (che aumentano all'aumentare dei job).

L'andamento dei modelli e degli algoritmi con α = 0,5 (Tabella 19) è molto simile al caso precedente come andamenti delle F.O, dei tempi e degli ottimi. Questi sono solo globalmente peggiorati. Il modello 1 base ora fatica a trovare ottimi anche per istanze piccole (20 job e 2 macchine), mentre il modello base 2 riesce a trovare solo un ottimo in meno rispetto al caso precedente (38 per il modello 1, 45 per il modello 2). I modelli di assegnazione peggiorano entrambi di 5 ottimi (ora rispettivamente 54 e 59), con rispettivamente 34 e 38 ottimi "reali", mentre, per quanto riguarda gli andamenti delle F.O. e dei tempi di calcolo restano valide le osservazioni fatte per il caso precedente.

[12] Ricordando che gli scostamenti vengono calcolati solo se si ha a disposizione la soluzione ottima dell'euristica e la soluzione ottima del problema di partenza (trovata con il modello 1 o il modello 2 o con i due algoritmi per T). Gli ottimi iniziali sono stati trovati maggiormente per istanze piccole, e quasi nessuno per istanze grandi. Quindi gli scostamenti con istanze piccole si sono potuti calcolarli facilmente e sono contenuti, anche perché sono poche le variabili assegnate in partenza. Con istanze grandi, lo scostamento, essendo la percentuale di assegnazione molto maggiore (per 100 job raggiunge il 50%), potrebbero essere molto maggiori, ma non si hanno gli ottimi di partenza per poter fare il confronto. Lo stesso accade per il calcolo del numero di ottimi uguali tra l'euristica e il problema di partenza, essendo che con l'euristica si hanno a disposizione molti ottimi che non possiamo confrontare (Tabella 4: assegnazione).

In tutte e quattro le versioni i "picchi" delle funzioni obiettivo sono ancora in corrispondenza delle lavorazioni affidate a 2 macchine.

Infine, con la Tabella 9, si studia il comportamento dei modelli base e delle due euristiche di assegnazione al variare di α. Si nota un peggioramento di tutti e quattro in termini di soluzioni ottime trovate, con un nettissimo calo da α = 0,1 a α = 0,3, mentre una diminuzione più contenuta si osserva tra gli ultimi due valori si α. Anche le F.O. obiettivo aumentano, ma non proporzionalmente rispetto alla discesa degli ottimi (quindi si suppone che in molti casi i modelli siano vicini alla soluzione ottima, ma non riescano a trovarla in tempo), mentre i tempi peggiorano molto (peggiorano sia i tempi di calcolo quando vengono trovate le soluzioni ottime, sia quando non vengono trovate, essendo che il solver ne trova di meno saranno tutti spostati verso il massimo). Probabilmente se si aumentasse il tempo massimo e/o la percentuale di assegnazione proporzionalmente all'aumento della difficoltà del problema da risolvere, i numeri di ottimi rimarrebbero più simili tra i vari livelli di α. Con α = 0,1 si hanno molte lavorazioni in termini di tempo di completamento e tempo di setup uguali tra di loro, quindi molto più facili da trattare, mentre aumentando α, aumenta la diversità reciproca delle lavorazioni, rendendo più difficile trovare la schedulazione ottima.

Inoltre, come già osservato con α = 0,1, tutte le versioni peggiorano al diminuire del numero di macchine e all'aumentare del numero di job. Questo si può vedere facilmente dai grafici di pagina 42 (dalla Figura 35: T medi a=0.3 in avanti). Si vede che l'andamento delle funzioni obiettivo è molto simile, mentre quello dei tempi è opposto, tra α = 0,1 e gli α successivi. Nel primo caso l'andamento è "basso" con dei picchi in alto in corrispondenza di m = 2, e delle "esplosioni" dei modelli base. nel secondo caso, l'andamento è molto più "alto" (a livello dei tempi massimi dati) con delle discese o dei picchi in basso all'aumentare del numero di macchine. Questo evidenzia la difficoltà maggiore del solver all'aumentare di α.

Un'ultima osservazione interessante riguardo all'algoritmo di assegnazione è che aumentando α diminuisce il numero di ottimi "reali" e locali trovati, ma il numero di essi che coincidono con i modelli base è maggiore (α = 0,1 26 per entrambi, α = 0,3 36 per assegnazione modello 1, 41 per assegnazione modello 2 e con α = 0,5 34 e 38 rispettivamente). In termini percentuali si noterebbe ancora di più l'incremento.

Quindi l'algoritmo di assegnazione all'aumentare di α peggiora le sue prestazioni, ma quando trova delle soluzioni ottime queste sono più vicine a quelle del problema di partenza. Questo vuol dire che il concetto di fondo su cui si basa l'algoritmo, per α grandi, è più solido, ovvero si avvicina di più alle disposizioni (schedule) dei job che effettuerebbe il solver se dovesse assegnare lui tutte le lavorazioni.

3. 3 Tabelle aggregate

Di seguito si trovano le tabelle con i dati aggregati, per avere un'idea d'insieme dell'andamento dei modelli. Le prime sono riferite per α = 0,1 e confrontano le sei varianti all'aumentare di m, aggregando i dati per n e ρ. L'ultima tabella confronta, per ρ = 1, l'andamento complessivo dei modelli e dell'algoritmo di assegnazione al variare di α, quindi aggregando per m e n.

Di seguito con numero ottimi per l'euristica di assegnazione si intende gli ottimi trovati relativi al problema semplificato, non gli ottimi coincidenti con i modelli base.

Infine, con $\sum F.O.$, $\overline{F.O.}$, $\sum t$, $\bar{t}$ si intende rispettivamente la somma aggregata della somma delle funzioni obiettivo, la somma delle funzioni obiettivo medie, la somma aggregata della somma dei tempi di calcolo e la somma dei tempi di calcolo medi.

TABELLA 5: COMPARAZIONE MODELLI, ALFA =0. 1, M = 2, DATI AGGREGATI PER N E P

m = 2	base		algo T		assegnazione	
	mod 1	mod 2	mod 1	mod 2	mod 1	mod 2
num. Ottimi	35	34	42	28	51	54
∑F.O.	207824	120117	66035	62361	75326	73521
$\overline{F.O.}$	41565	24023,4	14651	14597	15065	14704
∑t	13844	13791,90	11564	11456	12339	12267
$\bar{t}$	2769	2758,38	2655	2610	2468	2453

TABELLA 6: COMPARAZIONE MODELLI, ALFA = 0.1, M = 4, DATI AGGREGATI PER N E P

m = 4	base		algo T		assegnazione	
	mod 1	mod 2	mod 1	mod 2	mod 1	mod 2
num. Ottimi	46	48	45	49	72	73
∑F.O.	128963	70744	26353	26453	33343	33464
$\overline{F.O.}$	25793	14148,8	6725	6753	6669	6693
∑t	12680	12227,91	8970	8870	6688	6528
$\bar{t}$	2536	2445,58	2452	2434	1338	1306

TABELLA 7: COMPARAZIONE MODELLI, ALFA = 0.1, M = 6, DATI AGGREGATI PER N E P

m = 6	base		algo T		assegnazione	
	mod 1	mod 2	mod 1	mod 2	mod 1	mod 2
num. Ottimi	50	54	46	46	75	76
∑F.O.	100256	39327	20831	20852	21932	21928
$\overline{F.O.}$	20051	7865,4	4439	4444	4386	4386
∑t	12161	12089,74	11113	11204	5841	4950
$\bar{t}$	2432	2417,95	2383	2401	1168	990

TABELLA 8: COMPARAZIONE MODELLI, ALFA = 0.1, M = 6, DATI AGGREGAI PER N E P

m = 10	base		algo T		assegnazione	
	mod 1	mod 2	mod 1	mod 2	mod 1	mod 2
num. Ottimi	50	52	46	48	71	71
∑F.O.	59231	12486	12454	12444	12468	12466
$\overline{F.O.}$	11846,2	2497,2	2900,4	2488,8	2493,6	2493,2
∑t	9596,952	8619,169	9322,282	8571,71	2506,77073	2171,54
$\bar{t}$	1919,39	1723,834	1864,456	1714,342	501,354145	434,308

TABELLA 9: COMPARAZIONE MODELLI, P = 1, DATI AGGREGATI PER N E

	Modello 1			Modello 1 algoritmo assegnazione			Modello 2			Modello 2 algoritmo assegnazione		
α	num. Ottimi	$\overline{F.O.}$	$\bar{t}$	num. Ottimi	$\overline{F.O.}$	$\bar{t}$	num. Ottimi	$\overline{F.O.}$	$\bar{t}$	num. Ottimi	$\overline{F.O.}$	$\bar{t}$
0,1	68	27178	2928,12	103	8849	940,97	74	13492	2629,7	104	8729	880,43
0,3	47	28948	3385,65	59	8790	3225,14	46	12446	3469,2	64	8865,4	3160,74
0,5	38	32409	3606,35	54	8910	3304,76	45	13640	3476,8	59	9036,6	3247,68

3.4 Tabelle estese

Di seguito si trovano le tabelle estese con i dati per le varie combinazioni di m, n, ρ e α. All'inizio, come anticipato sopra, si tratta in maniera specifica per α = 0.1 tutte le combinazioni di m, n e ρ per tutte e sei le varianti. Successivamente si analizzano gli α = 0.3 e α = 0.5, tralasciando lo studio di ρ, essendo che si attende un andamento simile per tutte le α. Inoltre, come già detto in fase di analisi dei risultati, si tralascia anche l'andamento del modello potenziato con l'algoritmo per la determinazione di T, essendo che si è osservato (come per α = 0.1) un andamento sostanzialmente simile al modello base (le istanze sono state comunque testate e sono sempre disponibili online, insieme a tutte le tabelle e i grafici, al sito: https://github.com/SimoneM02/Tesi-).

Inoltre, sono stati generati dei grafici per aiutare a visualizzare le prestazioni. Per la prima parte di tabelle sono stati creati quattro grafici, due che relazionano gli ottimi travati con rispettivamente i tempi medi di calcolo e le funzioni obiettivo medie al variare di ρ, aggregando su n; e due che confrontano senza aggregazione, i tempi medi di calcolo e le funzioni obiettivo medie. Nei primi due grafici con 1 si intende il modello base, con 2 il modello con algoritmo per T e con 3 il modello di assegnazione.

Per la seconda parte, ad ogni tabella sono stati associati due grafici, uno relativo all'andamento dei tempi di calcolo medi e uno relativo all'andamento delle funzioni obiettivo medie. I grafici sono stati generati anche per α = 0.1 (per ρ=1) anche se non è presente la tabella relativa (basta prendere le righe relative a ρ = 1 nelle tabelle precedenti. La tabella, comunque, si può trovare sul file Excel online, al sito già indicato in precedenza).

Nelle tabelle seguenti con $\sum F.O.$, $\sum t$ si intende la somma delle funzioni obiettivo e la somma dei tempi di calcolo. Con ottimi assoluti si intende il numero di ottimi uguale al modello base, ove siano stati trovati entrambi (quindi per l'assegnazione). Con scostamento max. il massimo scostamento trovato nelle istanze in questione tra le funzioni obiettivo ottime dell'euristica di assegnazione e del modello base, anche qua ovviamente solo nei casi che entrambe siano state trovate, infine, con $\overline{scost.}$ si intende la media degli scostamenti, sempre con le considerazioni precedenti.

Invece, nelle ultime due tabelle con la colonna ottimi/reali si intende come primo numero gli ottimi trovati, come secondo numero (dopo il trattino –) gli ottimi coincidenti con il modello base, e, nella colonna scost. come primo numero di intende lo scostamento massimo e come secondo (dopo il trattino -) la media degli scostamenti, con tutte le considerazioni precedenti.

Infine, in tutte le tabelle il numero di asterischi indica il numero di soluzioni che il solver non è riuscito a trovare. Di conseguenza nel calcolo dei tempi e delle funzioni obiettivo medi e delle somme queste istanze non sono state tenute in considerazione (comunque t sarà pari al massimo datogli per quell'istanza).

TABELLA 10: ALFA= 1, M=2, P = 1, 2, 3 MODELLO 1

	m =2			Modello 1			Modello 1 algoritmo T			Modello 1 algoritmo assegnazione					
	n	ρ	Num. Istanze	Num. Ottimi	∑F.O.	∑t	Num. Ottimi	∑F.O.	∑t	Num. Ottimi	Ottimi Ass.	∑F.O.	∑t	scost. Max	$\overline{scost.}$
	10	1	5	5	823	14,35	5	823	6,57	5	5	823	3,1	0	0
	10	2	5	5	892	18,77	5	892	10,7	5	4	893	4,17	1	1
	10	3	5	5	993	19,36	5	993	10,15	5	5	993	4,36	0	0
	20	1	5	5	1621	148,66	5	1621	71,12	5	4	1622	27,09	1	0,2
	20	2	5	5	1752	137,33	5	1752	136,39	5	4	1753	71,66	1	0,2
	20	3	5	4	1944	222,50	1	1952	261,00	5	4	1945	109,67	1	0,2
	30	1	5	3	2442	374,06	5	2436	293,18	5	4	2437	249,40	1	0,2
	30	2	5	2	2679	422,47	5	2646	278,19	5	3	2648	127,07	1	0,4
	30	3	5	1	3098	433,38	5	2925	269,46	5	2	2930	275,32	2	1
	50	1	5	0	4364	1000,71	1	4023	928,60	2	-	4003	748,79	-	-
	50	2	5	0	4737	1002,65	0	4337	1000,46	2	-	4331	799,79	-	-
	50	3	5	0	5171	1001,19	0*	3851	800,33	1	-	4703	991,99	-	-
	75	1	5	0	23630	1502,08	0	6118	1500,68	0	-	6088	1500,91	-	-
	75	2	5	0	25688	1502,46	0*	5282	1200,54	0	-	6588	1500,65	-	-
	75	3	5	0	24276	1510,00	0	7280	1500,84	0	-	7237	1500,73	-	-
	100	1	5	0	31754	1508,17	0	8091	1501,16	1	-	8048	1421,89	-	-
	100	2	5	0	34247	1509,89	0**	5236	900,77	0	-	8702	1501,03	-	-
	100	3	5	0	37713	1515,89	0**	5777	901,11	0	-	9582	1501,73	-	-
totale			270	35	207824	13843,91	42	66035	11571,25	51	35	75326	12339,36		

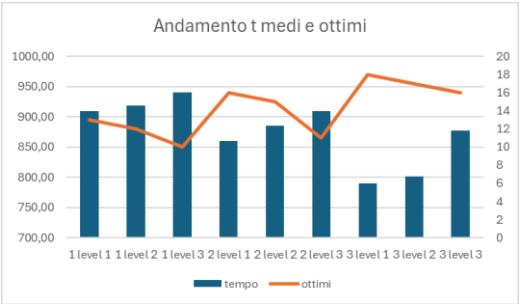


FIGURA 6: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 10

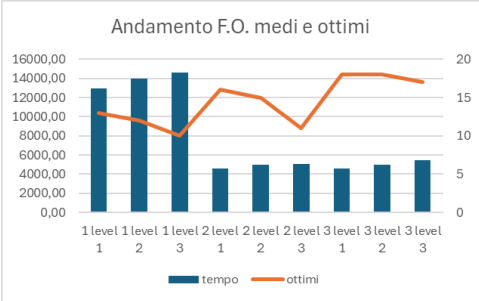


FIGURA 3: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 10

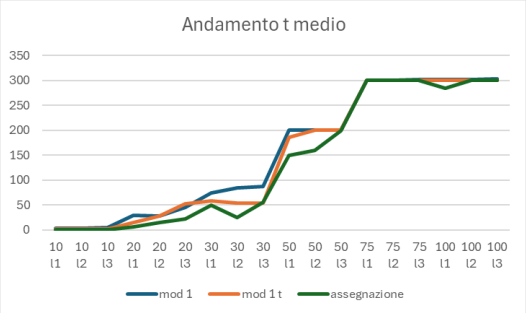


FIGURA 4: T MEDIO TAB. 10 CAMBIARE

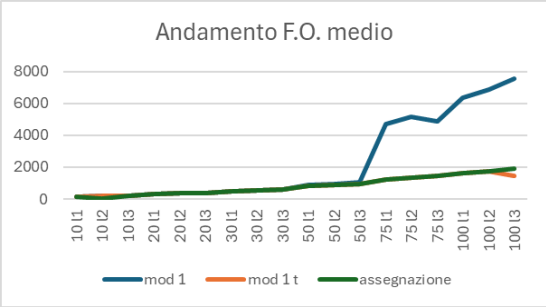


FIGURA 5: FO MEDI TAB. 10

TABELLA 11: ALFA= 1, M=2, P = 1, 2, 3 MODELLO 2

	m =2			Modello 2			Modello 2 algoritmo T			Modello 2 algoritmo assegnazione					
	n	ρ	Num. Istanze	Num. Ottimi	ΣF.O.	Σt	Num. Ottimi	ΣF.O.	Σt	Num. Ottimi	Ottimi Ass.	ΣF.O.	Σt	scost. Max	$\overline{scost.}$
	10	1	5	5	287	3,98	5	287	1,06	5	5	292	0,96	1	0
	10	2	5	5	312	3,81	5	312	0,92	5	4	322	0,77	2	1
	10	3	5	5	344	3,62	5	344	2,50	5	5	354	0,81	3	0
	20	1	5	5	1621	147,96	5	1621	86,96	5	4	1622	40,78	1	0,2
	20	2	5	4	1752	193,56	5	1752	81,21	5	4	1753	66,21	1	0,2
	20	3	5	5	1942	118,61	2	1947	240,18	5	3	1945	136,58	2	0,6
	30	1	5	2	2468	389,27	2**	1471	163,44	5	4	2437	192,07	1	0,2
	30	2	5	1	2673	484,86	3*	2117	213,17	5	3	2648	184,99	1	0,4
	30	3	5	2	3008	430,36	1***	1181	153,77	5	2	2930	194,91	2	1
	50	1	5	0	4257	1000,43	1	4026	888,57	2	-	4003	728,27	-	-
	50	2	5	0	4579	1000,53	0	4331	1000,39	2	-	4330	918,86	-	-
	50	3	5	0	6852	1000,98	0	3853	800,32	3	-	4699	1000,96	-	-
	75	1	5	0	10981	1500,93	0	6117	1500,58	1	-	6077	1296,42	-	-
	75	2	5	0	12872	1501,01	0	6615	1500,62	0	-	6569	1501,02	-	-
	75	3	5	0	14255	1501,35	0	7287	1516,06	0	-	7249	1500,62	-	-
	100	1	5	0	15918	1502,77	0	8087	1502,36	1	-	8043	1500,79	-	-
	100	2	5	0	17147	1502,91	0**	5236	900,71	0	-	8681	1501,04	-	-
	100	3	5	0	18849	1504,96	0**	5777	902,94	0	-	9567	1501,21	-	-
totale			270	34	120117	13791,90	28	62361	11455,77	54	34	73521	12267,29		

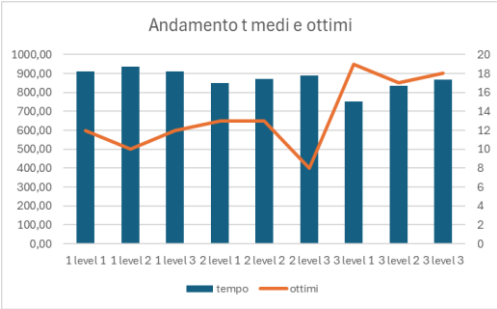


FIGURA 7: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 11

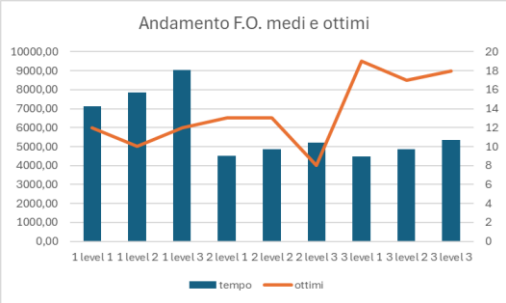


FIGURA 8: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 11

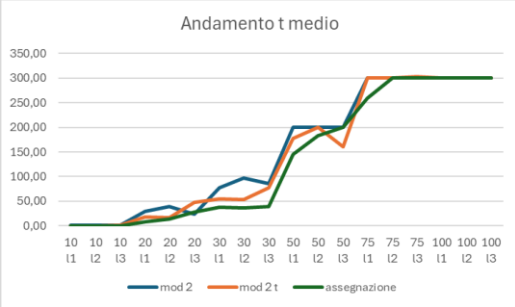


FIGURA 10: T MEDIO TAB. 11

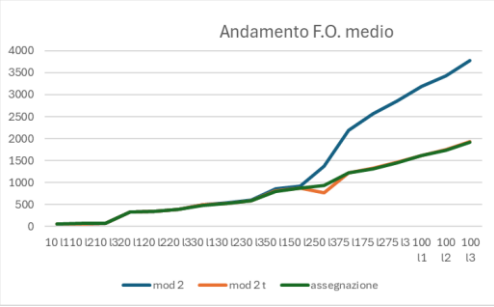


FIGURA 9: FO MEDIO TAB. 11

TABELLA 12: ALFA= 1, M=4, P = 1, 2, 3 MODELLO 1

	m =4			Modello 1			Modello 1 algoritmo T			Modello 1 algoritmo assegnazione					
	n	ρ	Num. Istanze	Num. Ottimi	ΣF.O.	Σt	Num. Ottimi	ΣF.O.	Σt	Num. Ottimi	Ottimi Ass.	ΣF.O.	Σt	scost. Max	$\overline{scost.}$
	10	1	5	5	419	7,69	5	419	2,68	5	0	428	1,62	2	1,8
	10	2	5	5	452	7,6	5	452	2,41	5	0	464	2,21	3	2,4
	10	3	5	5	485	10,66	5	485	4,08	5	0	496	2,01	4	2,2
	20	1	5	5	744	53,28	5	795	29,37	5	1	798	8,78	1	0,8
	20	2	5	5	795	65,04	5	877	38,45	5	2	883	11,25	1	0,6
	20	3	5	5	877	96,43	5	594	52,20	5	1	597	23,61	2	1,2
	30	1	5	5	1095	120,64	5	1095	44,19	5	0	1117	12,67	7	4,4
	30	2	5	5	1163	235,96	5	1163	139,60	5	0	1174	33,16	4	2,2
	30	3	5	4	1262	306,64	3	1265	255,11	5	1	1267	60,25	3	1,2
	50	1	5	0	1906	1000,55	2	1850	755,46	5	1	1831	21,63	4	1,8
	50	2	5	1	1983	862,20	0**	1186	600,12	5	0	1934	216,04	2	1,33
	50	3	5	1	2057	890,75	1*	1672	744,63	3	0	2093	725,86	3	3
	75	1	5	0	2967	1501,08	0*	2210	1200,34	5	-	2715	101,19	-	-
	75	2	5	0	3068	1501,50	0**	1746	900,20	0	-	2892	1500,40	-	-
	75	3	5	0	19769	1502,09	0***	1266	600,13	2	-	3106	1100,84	-	-
	100	1	5	0	28313	1504,87	0	3666	1500,46	5	-	3608	301,06	-	-
	100	2	5	0	29845	1506,24	0*	3101	1200,45	2	-	3823	1065,25	-	-
	100	3	5	0	31763	1506,98	0**	2511	900,33	0	-	4117	1500,55	-	-
totale			270	46	128963	12680,22	45	26353	8970,20	72	6	33343	6688,38		

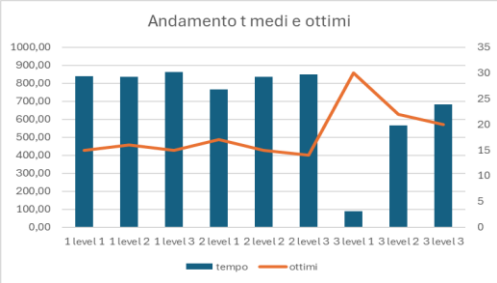


FIGURA 14: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 12

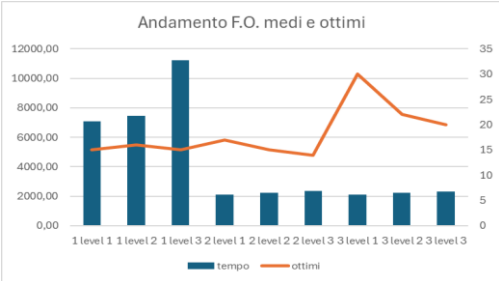


FIGURA 13: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 12

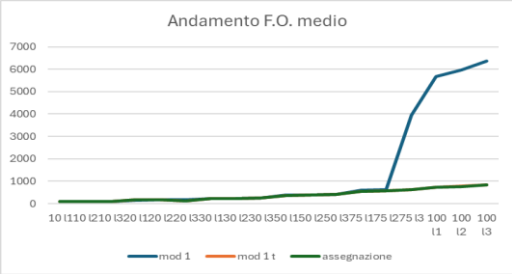


FIGURA 12: FO MEDI TAB. 12

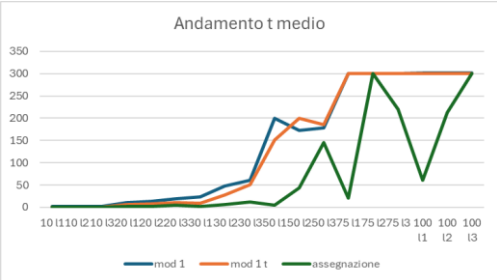


FIGURA 11: T MEDI TAB. 12

TABELLA 13: ALFA= 1, M=4, P = 1, 2, 3 MODELLO 2

	m =4			Modello 1			Modello 1 algoritmo T			Modello 1 algoritmo assegnazione					
	n	ρ	Num. Istanze	Num. Ottimi	∑F.O.	∑t	Num. Ottimi	∑F.O.	∑t	Num. Ottimi	Ottimi Ass.	∑F.O.	∑t	scost. Max	$\overline{scost.}$
	10	1	5	5	419	7,69	5	419	2,68	5	0	428	1,62	2	1,8
	10	2	5	5	452	7,6	5	452	2,41	5	0	464	2,21	3	2,4
	10	3	5	5	485	10,66	5	485	4,08	5	0	496	2,01	4	2,2
	20	1	5	5	744	53,28	5	795	29,37	5	1	798	8,78	1	0,8
	20	2	5	5	795	65,04	5	877	38,45	5	2	883	11,25	1	0,6
	20	3	5	5	877	96,43	5	594	52,20	5	1	597	23,61	2	1,2
	30	1	5	5	1095	120,64	5	1095	44,19	5	0	1117	12,67	7	4,4
	30	2	5	5	1163	235,96	5	1163	139,60	5	0	1174	33,16	4	2,2
	30	3	5	4	1262	306,64	3	1265	255,11	5	1	1267	60,25	3	1,2
	50	1	5	0	1906	1000,55	2	1850	755,46	5	1	1831	21,63	4	1,8
	50	2	5	1	1983	862,20	0**	1186	600,12	5	0	1934	216,04	2	1,33
	50	3	5	1	2057	890,75	1*	1672	744,63	3	0	2093	725,86	3	3
	75	1	5	0	2967	1501,08	0*	2210	1200,34	5	-	2715	101,19	-	-
	75	2	5	0	3068	1501,50	0**	1746	900,20	0	-	2892	1500,40	-	-
	75	3	5	0	19769	1502,09	0***	1266	600,13	2	-	3106	1100,84	-	-
	100	1	5	0	28313	1504,87	0	3666	1500,46	5	-	3608	301,06	-	-
	100	2	5	0	29845	1506,24	0*	3101	1200,45	2	-	3823	1065,25	-	-
	100	3	5	0	31763	1506,98	0**	2511	900,33	0	-	4117	1500,55	-	-
totale			270	46	128963	12680,22	45	26353	8970,20	72	6	33343	6688,38		

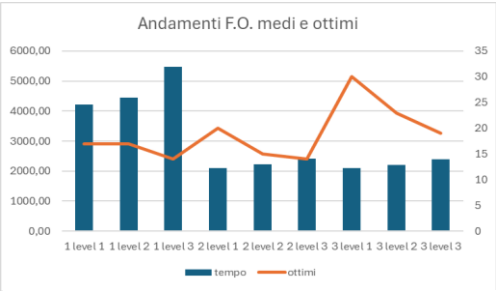


FIGURA 18: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 13

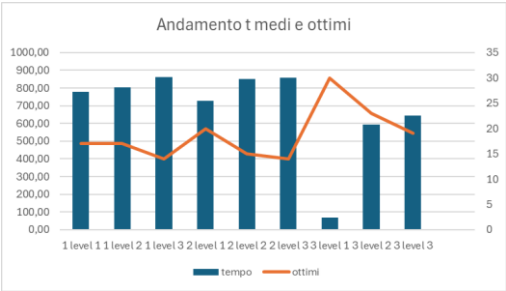


FIGURA 17 : FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 13

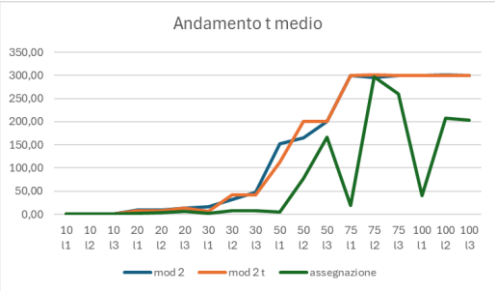


FIGURA 16: T MEDI TAB. 13

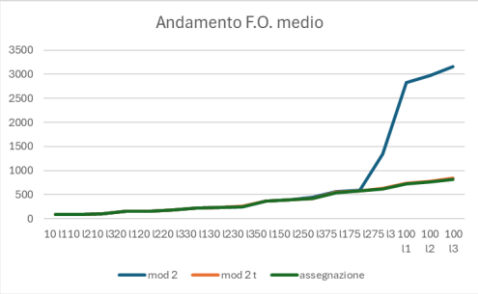


FIGURA 15: FO MEDI TAB. 13

TABELLA 14: ALFA= 1, M=6, P = 1, 2, 3 MODELLO 1

	m =6			Modello 1			Modello 1 algoritmo T			Modello 1 algoritmo assegnazione					
	n	ρ	Num. Istanze	Num. Ottimi	ΣF.O.	Σt	Num. Ottimi	ΣF.O.	Σt	Num. Ottimi	Ottimi Ass.	ΣF.O.	Σt	scost. Max	$\overline{scost.}$
	10	1	5	5	287	7,5	5	287	1,25	5	0	293	1,98	2	1,2
	10	2	5	5	318	6,16	5	318	1,2	5	0	328	0,75	2	2
	10	3	5	5	344	6,58	5	344	2,3	5	0	355	1,45	3	2
	20	1	5	5	524	31,73	5	524	10,062	5	0	540	3,80	4	3,2
	20	2	5	5	548	40,86	5	548	14,109	5	0	564	6,28	4	3,2
	20	3	5	5	593	94,09	4	593	112,082	5	0	600	22,52	2	1,4
	30	1	5	5	723	86,09	5	723	32,509	5	1	727	27,15	1	0,8
	30	2	5	5	772	108,67	5	772	139,124	5	1	777	17,50	2	1
	30	3	5	3	837	415,27	0	847	500,177	5	0	841	196,06	2	1
	50	1	5	5	1180	549,62	5	1180	258,911	5	0	1201	21,91	5	4,2
	50	2	5	2	1252	793,37	2	1262	839,540	5	0	1252	33,00	1	1,25
	50	3	5	0	1383	1000,61	0*	1100	800,126	0	-	1364	1000,21	-	-
	75	1	5	0	1847	1501,08	0	1797	1500,442	5	1	1766	140,62	1	0,5
	75	2	5	0	1946	1501,16	0	1894	1500,317	5	-	1852	616,83	-	-
	75	3	5	0	2140	1501,55	0	2048	1500,284	0	-	2013	1500,37	-	-
	100	1	5	0	27352	1504,24	0	2417	1500,423	5	-	2349	63,39	-	-
	100	2	5	0	28294	1504,54	0	2540	1500,366	5	-	2455	686,49	-	-
	100	3	5	0	29916	1508,33	0**	1637	900,262	0	-	2655	1500,31	-	-
totale			270	50	100256	12161,45	46	20831	11113,484	75	3	21932	5840,64		

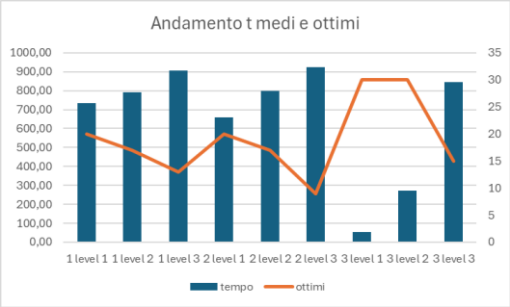


FIGURA 22: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 14

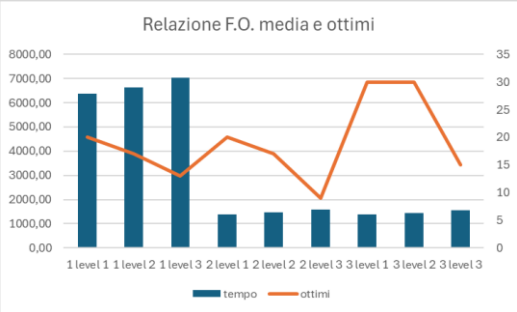


FIGURA 19: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 14

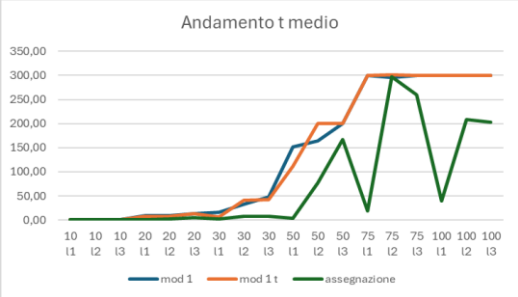


FIGURA 20: T MEDI TAB. 14

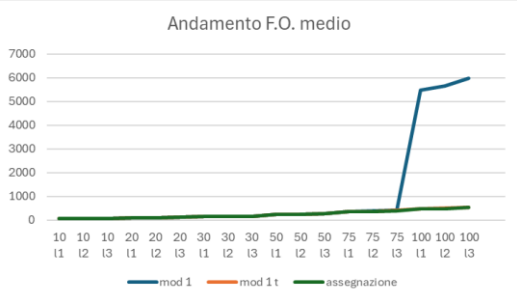


FIGURA 21: FO MEDI TAB. 14

TABELLA 15: ALFA= 1, M=6, P = 1, 2, 3 MODELLO 2

	m =6			Modello 2			Modello 2 algoritmo T			Modello 2 algoritmo assegnazione					
	n	ρ	Num. Istanze	Num. Ottimi	ΣF.O.	Σt	Num. Ottimi	ΣF.O.	Σt	Num. Ottimi	Ottimi Ass.	ΣF.O.	Σt	scost. Max	$\overline{scost.}$
	10	1	5	5	287	3,98	5	287	1,06	5	0	292	0,96	1	1
	10	2	5	5	312	3,81	5	312	0,92	5	0	322	0,77	2	2
	10	3	5	5	344	3,62	5	344	2,50	5	0	354	0,81	3	2
	20	1	5	5	524	19,92	5	524	9,10	5	0	540	3,97	4	3,2
	20	2	5	5	548	25,07	5	548	10,91	5	0	564	4,61	4	3,2
	20	3	5	5	593	70,15	4	594	111,43	5	0	600	20,21	2	1,4
	30	1	5	5	723	60,22	5	723	15,64	5	1	727	22,28	1	0,8
	30	2	5	5	772	75,02	5	772	101,72	5	1	777	10,51	2	1
	30	3	5	3	845	408,54	1	853	471,74	5	0	841	121,22	2	1
	50	1	5	5	1180	280,34	5	1180	427,59	5	0	1201	23,80	5	4,2
	50	2	5	4	1245	578,16	1	1254	849,94	5	0	1252	34,06	1	1,25
	50	3	5	0	1375	1000,32	0*	1113	800,11	1	-	1366	816,85	-	-
	75	1	5	2	1815	1315,07	0	1811	1500,32	5	1	1766	127,21	1	0,5
	75	2	5	0	1953	1500,60	0	1895	1500,21	5	-	1852	485,38	-	-
	75	3	5	0	2176	1500,72	0	2056	1500,24	0	-	2015	1500,26	-	-
	100	1	5	0	2480	1501,31	0	2419	1500,33	5	-	2349	86,47	-	-
	100	2	5	0	7284	1501,04	0	2536	1500,26	5	-	2455	190,70	-	-
	100	3	5	0	14871	2241,84	0**	1631	900,17	0	-	2655	1500,32	-	-
totale			270	54	39327	12089,74	46	20852	11204,18	76	3	21928	4950,39		

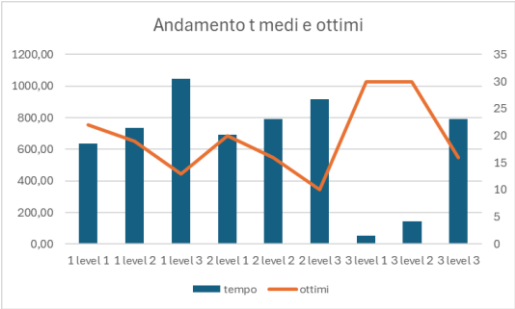


FIGURA 26: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N. TAB. 15

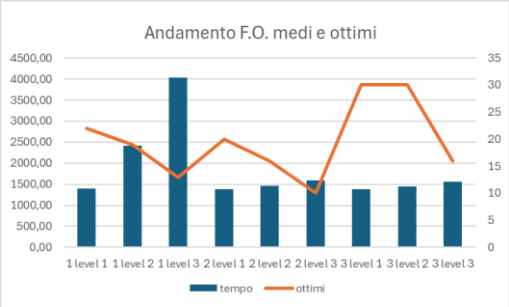


FIGURA 25: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 15

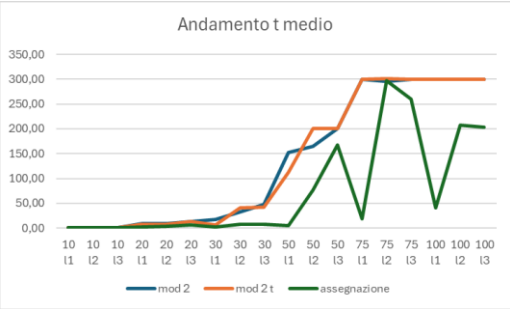


FIGURA 24: T MEDI TAB. 15

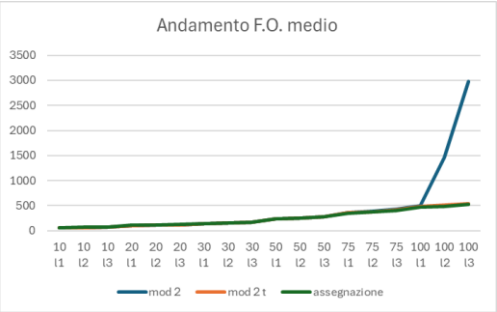


FIGURA 23: FO MEDI TAB. 15

TABELLA 16: ALFA= 1, M=10, P = 1, 2, 3 MODELLO 1

	m =10			Modello 1			Modello 1 algoritmo T			Modello 1 algoritmo assegnazione					
	n	ρ	Num. Istanze	Num. Ottimi	∑F.O.	∑t	Num. Ottimi	∑F.O.	∑t	Num. Ottimi	Ottimi Ass.	∑F.O.	∑t	scost. Max	$\overline{scost.}$
	20	1	5	5	295	24,35	5	295	1,88	5	3	297	1,99	1	0,4
	20	2	5	5	352	26,60	5	352	2,84	5	0	360	1,19	2	1,6
	20	3	5	5	370	34,64	5	370	11,07	5	0	379	5,75	2	1,8
	30	1	5	5	429	43,07	5	429	3,52	5	4	430	2,54	1	0,2
	30	2	5	5	486	60,78	5	486	38,17	5	0	498	3,64	3	2,4
	30	3	5	5	506	100,45	5	506	130,30	5	0	517	24,35	3	2,2
	50	1	5	5	694	153,40	5	694	15,99	5	3	696	6,86	1	0,4
	50	2	5	5	760	339,59	4	762	607,73	5	0	765	138,08	1	1
	50	3	5	2	787	916,59	1	794	926,04	3	0	802	512,22	3	2,5
	75	1	5	5	1023	496,89	5	1023	118,16	5	0	1056	8,66	7	6,6
	75	2	5	3	1108	1384,09	1	1119	1465,46	5	0	1117	13,42	4	2,75
	75	3	5	0	1195	1501,09	0	1180	1500,20	3	-	1168	1126,27	-	-
	100	1	5	0	1456	1506,49	0	1372	1500,36	5	0	1370	27,89	2	1,5
	100	2	5	0	27428	1504,27	0	1499	1500,31	5	-	1465	284,27	-	-
	100	3	5	0	22342	1504,66	0	1573	1500,24	5	-	1548	349,62	-	-
totale			225	50	59231	9596,95	46	12454	9322,28	71	10	12468	2506,77		

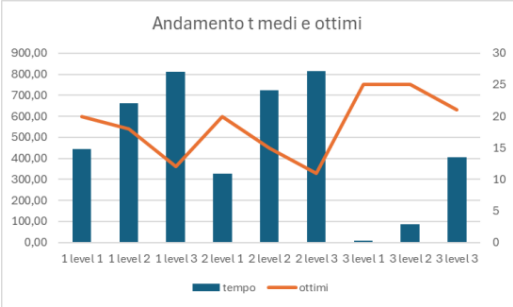


FIGURA 30: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 16

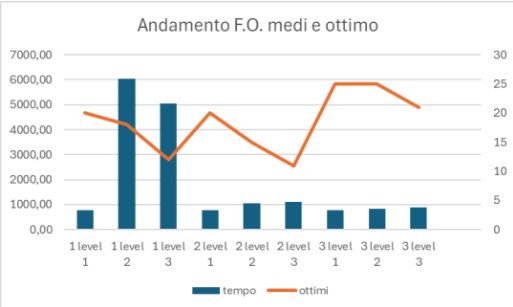


FIGURA 29: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 16

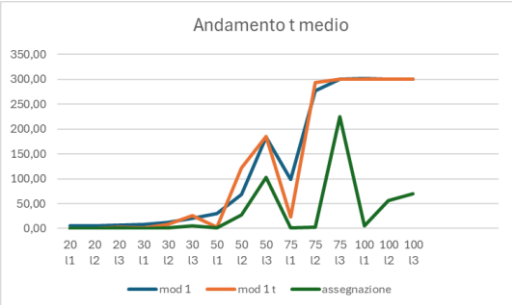


FIGURA 28: T MEDI TAB. 16

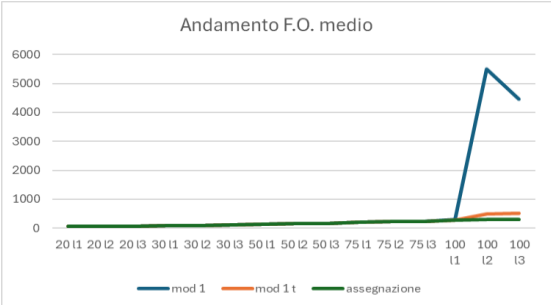


FIGURA 27: FO MEDI TAB. 16

TABELLA 17: ALFA= 1, M=10, P = 1, 2, 3 MODELLO 2

	m =10			Modello 2			Modello 2 algoritmo T			Modello 2 algoritmo assegnazione					
	n	ρ	Num. Istanze	Num. Ottimi	∑F.O.	∑t	Num. Ottimi	∑F.O.	∑t	Num. Ottimi	Ottimi Ass.	∑F.O.	∑t	scost. Max	$\overline{scost.}$
	20	1	5	5	295	12,48	5	295	1,85	5	3	297	1,61	1	0,4
	20	2	5	5	352	13,52	5	352	3,13	5	0	360	1,70	2	1,6
	20	3	5	5	370	17,92	5	370	11,72	5	0	379	5,86	2	1,8
	30	1	5	5	429	20,82	5	429	3,95	5	4	430	2,85	1	0,2
	30	2	5	5	486	36,13	5	486	29,45	5	0	498	8,49	3	2,4
	30	3	5	5	506	53,80	5	506	77,24	5	0	517	44,11	3	2,2
	50	1	5	5	694	78,34	5	694	11,01	5	3	696	5,42	1	0,4
	50	2	5	5	760	262,45	4	762	747,51	5	0	764	62,55	1	1
	50	3	5	1	798	955,89	0	804	1000,10	4	0	803	469,88	3	2,5
	75	1	5	5	1023	282,97	5	1023	102,51	5	0	1056	8,45	7	6,6
	75	2	5	3	1110	1253,24	1	1124	1399,12	5	0	1117	10,42	4	2,75
	75	3	5	0	1177	1500,52	0	1174	1500,16	2	-	1166	1141,78	-	-
	100	1	5	3	1369	1129,05	3	1363	683,45	5	0	1370	24,44	2	1,5
	100	2	5	0	1518	1500,86	0	1497	1500,32	5	-	1465	122,87	-	-
	100	3	5	0	1599	1501,17	0	1565	1500,21	5	-	1548	261,10	-	-
totale			225	52	12486	8619,17	48	12444	8571,71	71	10	12466	2171,54		

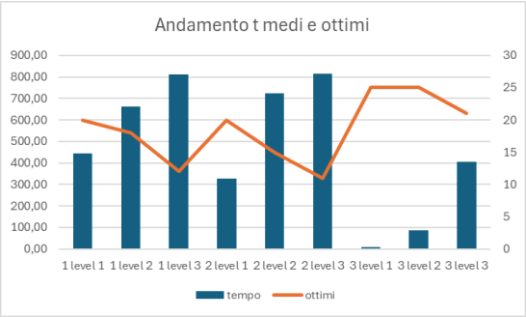


FIGURA 34: T MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 17

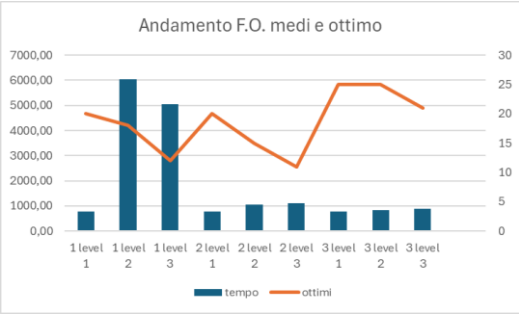


FIGURA 33: FO MEDI E OTTIMI AL VARIARE DI P, AGGREGATO SU N, TAB. 17

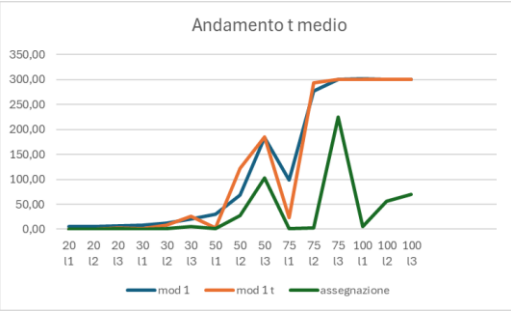


FIGURA 32: T MEDI TAB. 17

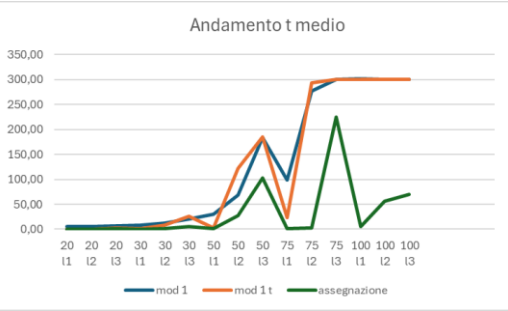


FIGURA 31: FO MEDI TAB. 17

TABELLA 18: P = 1, ALFA = 0.3

α = 0,3			Modello 1								Modello 2				Modello 2 algoritmo assegnazione				
n	m	Num. Istanze	Num. Ottimi	∑F.O.	∑t	Ottimi/reali	∑F.O.		∑t	scost	Num. Ottimi	∑F.O.	∑t	Ottimi/reali	∑F.O.		∑t		scost
10	2	5	5	789	15,31	5 - 4	790	158	7,11	1 - 0,2	5	789	9,42	5 - 4	790	158	8,97	1,79	1 - 0,2
10	4	5	5	400	9,25	5 - 3	405	81	4,44	2 - 0,8	5	400	4,78	5 - 3	405	81	3,52	0,70	2 - 0,8
10	6	5	5	275	7,17	5 - 3	277	55,4	2,76	1 - 0,4	5	275	4,10	5 - 3	277	55,4	3,35	0,67	1 - 0,4
20	2	5	3	1615	241,93	4 - 1	1612	322	221,16	1 - 0,5	3	1612	209,15	5 - 1	1610	322	122,26	24,45	1 - 0,5
20	4	5	5	739	124,19	5 - 1	743	149	65,54	1 - 0,8	5	739	90,54	5 - 1	743	149	44,94	8,99	1 - 0,8
20	6	5	5	476	45,41	5 - 3	479	95,8	18,58	2 - 0,6	5	476	36,11	5 - 3	479	95,8	16,98	3,40	2 - 0,6
20	10	5	5	296	27,29	5 - 3	298	59,6	5,38	1 - 0,4	5	296	17,59	5 - 3	298	59,6	5,77	1,15	1 - 0,4
30	2	5	0	2469	500,45	0	2425	485	500,16	-	0	2444	500,24	2	1920	384	336,22	67,24	-
30	4	5	0	1100	500,30	2	1106	221	414,66	-	0	1103	500,27	2	1109	222	374,58	74,92	-
30	6	5	5	695	365,29	5 - 4	696	139	136,74	1 - 0,2	4	699	280,73	5 - 4	696	139	115,81	23,16	1 - 0,2
30	10	5	5	431	84,72	5 - 5	431	86,2	15,07	-	5	431	57,59	5 - 5	431	86,2	21,54	4,31	-
50	2	5	0	10721	1000,97	0	3965	793	1000,26	-	0	4131	1000,40	0	4453	891	1000,31	200,06	-
50	4	5	0	1955	1000,85	1	1880	376	984,42	-	0	1923	1000,40	1	2075	415	972,53	194,51	-
50	6	5	0	1200	1000,51	0	1175	235	1000,14	-	0	1187	1000,28	2	1317	263	834,42	166,88	-
50	10	5	4	706	780,88	5 - 0	720	144	156,53	4 - 3	4	706	616,64	5 - 0	720	144	274,25	54,85	4 - 3
75	2	5	0	23322	1501,87	0	5951	1190	1500,64	-	0	9525	1500,89	0	5969	1194	1500,41	300,08	-
75	4	5	0	6618	1501,87	0	2737	547	1500,28	-	0	2788	1500,55	2	2735	547	1474,80	294,96	-
75	6	5	0	1851	1501,66	1	1758	352	1448,73	-	0	1784	1500,53	1	1759	352	1483,81	296,76	-
75	10	5	0	1100	1501,10	3	1064	213	1247,52	-	0	1052	1500,42	3	1067	213	1344,21	268,84	-
100	2	5	0	31442	1508,19	0	7985	1597	1500,83	-	0	15716	1503,12	0	7986	1597	1500,67	300,13	-
100	4	5	0	23715	1504,19	0	3691	738	1500,36	-	0	8055	1510,72	1	3699	740	1363,86	272,77	-
100	6	5	0	26968	1503,30	1	2340	468	1469,32	-	0	4676	1500,81	0	2361	472	1500,29	300,06	-
100	10	5	0	6672	1501,90	2	1421	284	1425,05	-	0	1423	1500,82	0	1428	286	1500,22	300,04	-
totale		460	47	145555	17728,61	59 - 36	43949	8790	16125,69		46	62230	17346,10	64 - 41	44327	8865	15803,70	3160,74	-

TABELLA 19: P = 1, ALFA = 0.5

α = 0,5				Modello 1			Modello 1 algoritmo assegnazione				Modello 2								
n	m	Num. Istanze	Num. Ottimi	∑F.O.	∑t	Ottimi/reali	∑F.O.		∑t	scost	Num. Ottimi	∑F.O.	∑t	Ottimi/reali	∑F.O.		∑t	$\bar{t}$	scost
10	2	5	5	856	22,71	5 - 5	856	171	9,57	-	5	856	22,32	5 - 5	856	171	7,87	1,57	-
10	4	5	5	382	9,65	5 - 5	382	76,4	5,38	-	5	382	5,62	5 - 5	382	76,4	5,68	1,14	-
10	6	5	5	252	7,02	5 - 2	255	51	2,15	1 - 0,6	5	252	3,38	5 - 2	255	51	2,66	0,53	1 - 0,6
20	2	5	0	1607	300,23	4 -1	1596	319	157,81	1 - 0,5	2	1598	258,05	4 - 1	1595	319	164,50	32,90	1 - 0,5
20	4	5	5	724	129,43	5 - 5	724	145	65,30	1 - 0,8	5	724	116,10	5 - 5	724	145	74,59	14,92	1 - 0,8
20	6	5	5	460	49,29	5 - 5	460	92	21,90	-	5	460	44,30	5 - 5	460	92	21,24	4,25	-
20	10	5	5	290	29,26	5 - 3	292	58,4	7,48	1 - 0,4	5	290	19,22	5 - 3	292	58,4	6,31	1,26	1 - 0,4
30	2	5	0	2499	500,44	1	2399	480	426,96	-	0	2449	500,23	1	2402	480	441,34	88,27	-
30	4	5	0	1086	500,32	2 - 0	1084	217	444,17	-	0	1089	500,23	3 - 0	1083	217	472,83	94,57	-
30	6	5	2	689	401,92	5 - 2	686	137	108,31	1 - 0,33	3	689	362,14	5 - 2	686	137	70,87	14,17	1 - 0,33
30	10	5	5	425	100,66	5 -4	426	85,2	23,84	1 - 0,2	5	425	43,25	5 - 4	426	85,2	20,77	4,15	1 - 0,2
50	2	5	0	11052	1000,84	0	3989	798	1000,29	-	0	4218	1000,48	0	4503	901	1000,31	200,06	-
50	4	5	0	1961	1000,66	0	1936	387	1000,20	-	0	1909	1000,37	1	2118	424	1000,00	200,00	-
50	6	5	0	1134	1000,55	2	1119	224	858,50	-	1	1133	968,58	1	1124	225	862,11	172,42	-
50	10	5	1	717	948,89	5 - 0	723	145	384,96	2 - 2	2	718	740,38	5 - 0	723	145	276,32	55,26	2 - 2
75	2	5	0	23414	1502,82	0	6034	1207	1500,49	-	0	11726	1500,79	0	6056	1211	1500,47	300,09	-
75	4	5	0	18308	1506,00	0	2965	593	1500,33	-	0	3000	1502,80	0	2915	583	1500,30	300,06	-
75	6	5	0	1914	1501,83	0	1870	374	1500,23	-	0	1857	1501,67	1	1866	373	1379,84	275,97	-
75	10	5	0	1122	1500,99	0	1096	219	1500,18	-	1	1079	1412,30	2	1091	218	1428,20	285,64	-
100	2	5	0	31125	1506,70	0	8053	1611	1504,85	-	0	15631	1501,79	0	8036	1607	1501,24	300,25	-
100	4	5	0	28222	1505,33	0	3734	747	1500,44	-	0	13991	1501,31	0	3721	744	1500,39	300,08	-
100	6	5	0	27336	1503,88	0	2439	488	1500,30	-	0	2340	1500,68	0	2434	487	1500,31	300,06	-
100	10	5	0	6468	1502,34	0	1432	286	1500,18	-	1	1383	1378,24	0	1435	287	1500,26	300,05	-
totale		460	38	162043	18031,77	54 - 34	44550	8910	16523,81		45	68199	17384,21	59 - 38	45183	9037	16238,40	3247,68	

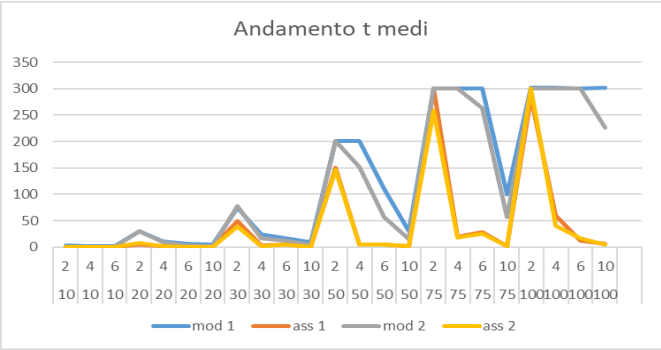
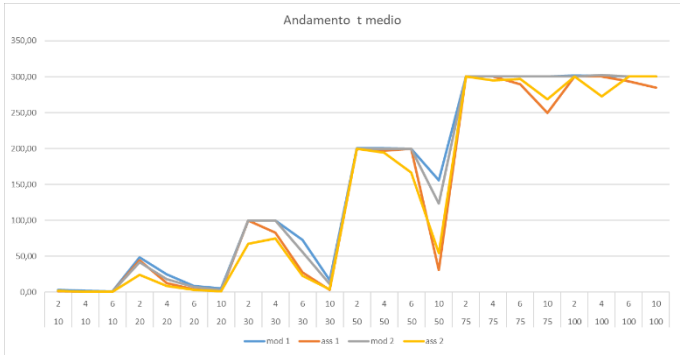


FIGURA 37: T MEDI A=0.1

FIGURA 35: T MEDI A=0.3



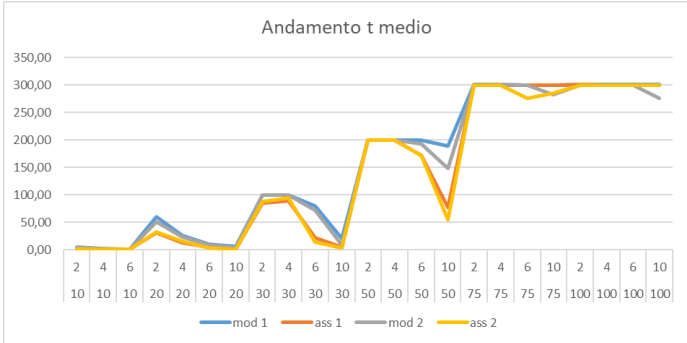


FIGURA 36: T MEDI A=0.5

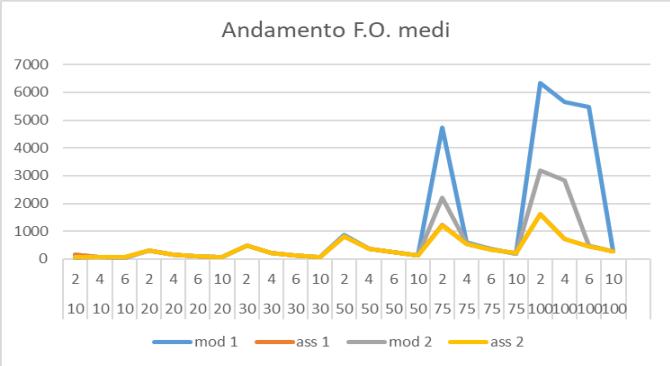
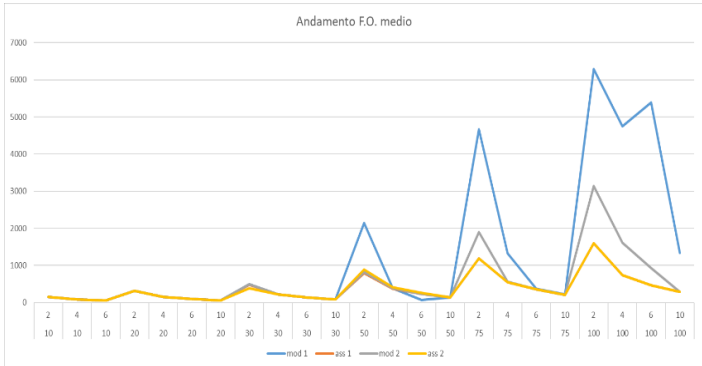


FIGURA 39: FO MEDI A=0.1

FIGURA 38: FO MEDI A=0.3



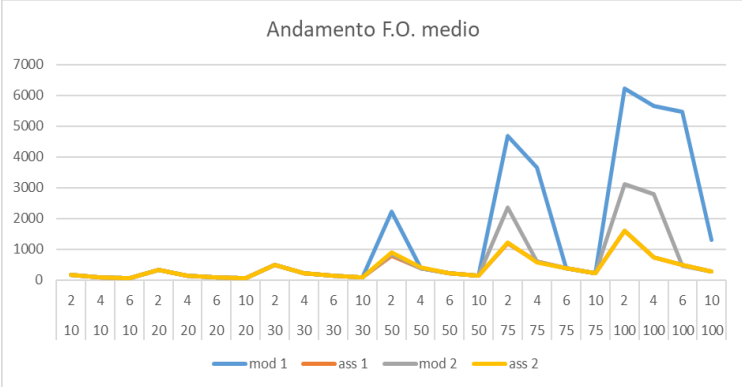


FIGURA 40: FO MEDI A=0.5

4 - Conclusioni

In questo elaborato è stato affrontato il problema della schedulazione su macchine parallele con un unico server per le operazioni di setup. Sono stati proposti due modelli di programmazione lineare per la risoluzione di questi problemi. Il secondo modello è un miglioramento del primo e va a dimezzare le variabili create e a ridurre i vincoli creati, migliorando le prestazioni del modello. Nel tentativo di aumentare ancora le performance è stato introdotto un semplice algoritmo per dare in input ai due modelli un orizzonte temporale più piccolo, evitando di creare inutili variabili. Infine è stato proposto un semplice algoritmo euristico di allocazione di una certa percentuale di job alle macchine in partenza da aggiungere ai modelli e all'algoritmo per trovare T. Quindi in tutto sono state sviluppate tre versioni per modello.

Questi sono stati testati su un largo numero di istanze con caratteristiche diverse, come il numero di macchine, il numero di lavorazioni, la lunghezza dei tempi di setup e la diversità reciproca tra i tempi di setup e di processo, e in base a questi parametri è stato dato ad un solver un tempo massimo per cercare di arrivare alla soluzione ottima.

I risultati sono stati aggregati in tabelle e valutati. Questi ultimi hanno evidenziato come i modelli base, soprattutto il modello 1, siano poco performanti, ovvero sono molto lontani da arrivare alla soluzione.

L'algoritmo per la determinazione del tempo arriva molto più vicino alle soluzioni, ma nei limiti di tempo dati non introduce grandi miglioramenti rispetto al modello base (questo non è lo stesso vero se si cambiano i limiti di tempo).

Come da aspettative, l'euristica di assegnazione è quella che ha ottenuto i risultati migliori. Con una semplice allocazione di alcuni job è stato possibili ridurre di molto i tempi e aumentare le soluzioni trovate, anche se queste si discostano di poco dalla soluzione ottima ottenibile con i modelli di partenza. Tuttavia, anche questo algoritmo mostra notevoli peggioramenti all'aumentare soprattutto di ρ e α e con $m = 2$. Anche qua le prestazioni potrebbero essere molto diverse aumentando il tempo limite o la percentuale di assegnazione.

In conclusione, questo elaborato mostra come diminuire il numero di variabili e di vincoli, andandole ad eliminare, oltre che dal modello in sé cambiando la formulazione, da dove non servono, come introducendo un orizzonte temporale più contenuto, si possano ottenere benefici. Inoltre, sviluppando un semplice algoritmo che toglie parte del lavoro al solver, le prestazioni sono molto migliorate. Quindi implementando algoritmi o modelli più sofisticati si possono sicuramente aumentare ancora di più le soluzioni ottime trovate e diminuire i tempi di calcolo, anche accontentandosi di una soluzione leggermente peggiorante.

Bibliografia

Aira, L. S. (2022). *Approximation Algorithms for Uniform Parallel Machine Scheduling through Mathematical Programming*. Tesi: Corso di laurea magistrale in Ingegneria Matematica, Politecnico di Torino.

Alfieri. (2023). *Programmazione e controllo della produzione*. McGraw-Hill Education.

Allahverdi, A. (2007). *Heuristics for the Two-Machine Scheduling Problem with a Single Server."* International Transactions in Operational Research 24 (6): 1347–1355. https://doi.org/10.1111/itor.2017.24.issue-6.

Bektur, G. a. (2019). *"A Mathematical Model and Heuristic Algorithms for An Unrelated Parallel Machine Scheduling Problem with Sequence-Dependent Setup Times, Machine Eligibility Restrictions and a Common Server."*. Computers & Operations Research 103:46–63. https://doi.org/10.1016/j.cor.2018.10.

Della Croce, T. (2010). *Elementi di Ricerca Operatica*. Società editrice Esculapio.

Elidrissi. (2020). *"Variable Neighborhood Search for Identical Parallel Machine Scheduling Problem with a Single Server."*. Variable Neighborhood Search, edited by Rachid Benmansour, Angelo Sifaleras, and Nenad Mladenović, 112–125. Cham, Switzerland: Springer International Publishing.

Elidrissi, A. M. (2018b). *"MIP Formulations for Identical Parallel Machine Scheduling Problem with Single Server."* . 4th International Conference on Optimization and Applications (ICOA), 1–6. s. https://doi.org/10.1051/matecconf/201820000001.

Elidrissi, A. R. (2021). *"Mathematical Formulations for the Parallel Machine Scheduling Problem with a Single Server."* . International Journal of Production Research 59 (20): 6166–6184. https://doi.org/10.1080/00207543.2020.1807637.

Glass, C. A. (2020). *"Scheduling for Parallel Dedicated Machines with a Single Server."*. Naval Research Logistics 47 (4): 304–328. https://doi.org/10.1002/(ISSN)1520-6750.

Hamzadayi, A. a. (2017). *"Modeling and Solving Static m Identical Parallel Machines Scheduling Problem with a Common Server and Sequence Dependent Setup Times."*. Computers & Industrial Engineering 106:287–298. https://doi.org/10.1016/j.cie.2017.02.013.

Jiang, Y. Q. (2015). *"Single-Server Parallel-Machine Scheduling with Loading and Unloading Times."*. Journal of Combinatorial Optimization 30 (2): 201–213. . https:// doi.org/10.1007/s10878-014-9727-z.

João Marcos Pereira Silva, A. S. (2023). *On time-indexed formulations for the parallel machine scheduling problem with a common server, Engineering Optimization*. https://doi.org/10.1080/0305215X.2023.2269847: DOI: 10.1080/0305215X.2023.2269847.

Kim, M.-Y. a. (2012). *"MIP Models and Hybrid Algorithm for Minimizing the Makespan of Parallel Machines Scheduling Problem with a Single Server."*. https://doi.org/10.1016/j.cor.2011.12.011: " Computers & Operations Research 39 (11): 2457–2468.

Koulamas, C. P. (2009). *"Scheduling Two Parallel Semiautomatic Machines to Minimize Machine Interference."*. Computers & Operations Research 23 (10): 945–956. https://doi.org/10.1016/0305-0548(96)00011-1.

Liu, G.-S. J.-J.-D. (2019). *"Approximate and Branch-and-Bound Algorithms for the Parallel Machine Scheduling Problem with a Single Server."*. Journal of the Operational Research Society 70 (9): 1554–1570. https://doi.org/10.1080/01605682.2018.1500976.

Potts, C. N. (2009). *"Fifty Years of Scheduling: A Survey of Milestones."*. https://doi.org/10.1057/jors.2009.2: Operational Research Society 60 (1): S41–S68.

Silva, J. M. (2021). *"Exact and Metaheuristic Approaches for Identical Parallel Machine Scheduling with a Common Server and Sequence-Dependent Setup Times."* . Journal of the Operational Research Society 72 (2): 444–457. https://doi.org/10.1080/01605682.2019.1671153.

Appendice A

Di seguito i codici dei due modelli e degli algoritmi in aggiunta.

A.1 Codice Modello 1

```
from mip import*
import time
//INPUT
T = sum(tj) +1
m = Model('Time-Indexed Formulations TIFs')
y =[[m.add_var(name='y({})({})'.format(i, t), var_type=BINARY) for t in range(T-
tj[i])] for i in range(J+JD)]

Cmax = m.add_var(name='Cmax')

for i in range(J):
   m+= xsum(y[i][t] for t in range(T-tj[i])) == 1, '(1b)'

for t in range(T-1):
   m+= xsum(y[i][s] for i in range(J) for s in range(max(0, (t-tj[i]+1)), min(t+1,
(T-tj[i])))) <= macchine, '(1c)'

for i in range(JD):
   m+= xsum(y[i+n][t] for t in range(T-sj[i]) if t < T-tj[i]) == 1, '(1d)'

for t in range(T-1):
   m+= xsum(y[i+n][s] for i in range(JD) for s in range(max(0, (t-sj[i]+1)),
min(t+1,(T-tj[i])))) <= 1, '(1e)'

for i in range(J):
    m+= Cmax >= xsum((t + tj[i])*y[i][t] for t in range(T-tj[i])), '(1f)'

for i in range(J):
    for t in range(T-tj[i]):
        m+= y[i][t] == y[i+n][t], '(1g)'

m.objective = minimize(Cmax)
start_time = time.time()
status = m.optimize(max_seconds=)
m.max_mip_gap = 0
end_time = time.time()
runtime = end_time - start_time
//OUTPUT
```

A.2 Codice Modello 2

```
from mip import*
import time
//INPUT
T = sum(tj) +1

m = Model('Time-Indexed Formulations TIFs')
y =[[m.add_var(name='y({})({})'.format(i, t), var_type=BINARY) for t in range(T-
tj[i])] for i in range(J)]

Cmax = m.add_var(name='Cmax')

w = [m.add_var(name='w({})'.format(t), var_type=INTEGER, lb=0, ub= macchine) for t
in range(T)]

for i in range(J):
    m+= xsum(y[i][t] for t in range(T-tj[i])) == 1, '(1b)'
for t in range(T-1):
    m+= xsum(y[i][s] for i in range(J) for s in range(max(0, t-tj[i]+1), min(t+1,
T-tj[i]))) + w[t] == macchine, '(2b)'
for t in range(T-1):
    m+= xsum(y[i][s] for i in range(J) for s in range(max(0, t-sj[i]+1), min(t+1,
T-tj[i]))) <= 1, '(2c)'
for i in range(J):
    m+= Cmax >= xsum((t + tj[i])*y[i][t] for t in range(T-tj[i])), '(1f)'

m.objective = minimize(Cmax)
start_time = time.time()
status = m.optimize(max_seconds=)
m.max_mip_gap = 0
end_time = time.time()
runtime = end_time - start_time
//OUTPUT
```

A.3 Codice algoritmo tempo

```
T = 0
Tm = list(0 for c in range(macchine))
contatorejob = 0
for j in range(0, (math.ceil(J/macchine)*macchine), macchine):
    for c in range(macchine):
        if contatorejob < J:
            Tm[c] += sj[contatorejob]
```

```
              for s in range(macchine):
                  if s != c:
                       if Tm[s] < Tm[c]:
                          Tm[s] = Tm[c]
          Tm[c] += pj[contatorejob]
     contatorejob += 1
T = max(Tm)+1
```

A.4 Codice algoritmo assegnazione

```
combined_data = list(zip(pj, sj, tj))
sorted_combined_data = sorted(combined_data, key=lambda x: (x[1], -x[0]))

s_pj = [data[0] for data in sorted_combined_data]
s_sj = [data[1] for data in sorted_combined_data
s_tj = [data[2] for data in sorted_combined_data]

 arr =

var_assegnare = math.ceil(arr * J)

tempo_server = 0
tempo_macchina = []
for a in range(macchine):
     tempo_macchina.append(0)

job_ass = 0

//INIZIALIZZAZIONE MODELLO

for mm in range(0, macchine*(math.ceil(var_assegnare/macchine)), macchine):
   for macch in range(macchine):
       if job_ass < var_assegnare:
           y[job_ass][max(tempo_server, tempo_macchina[macch])] = 1
           tempo_macchina[macch] = max(tempo_server, tempo_macchina[macch]) +
s_sj[job_ass]
            tempo_server =  tempo_macchina[macch]
            tempo_macchina[macch] += s_pj[job_ass]
            job_ass +=1
        tempo_server = max(tempo_server, min(tempo_macchina))
```